



Théorie des graphes

– Cours 2 –

Chapitre 2 : ARBRES ET ARBORESCENCE (1/1)

Dr. Benaliouche Houda

Faculté des nouvelles technologies

Houda.benaliouche@univ-constantine2.dz



Théorie des graphes

– Cours 2 –

Chapitre 2 : ARBRES ET ARBORESCENCE (1/1)

Dr. Benaliouche Houda

Faculté des **nouvelles technologies**

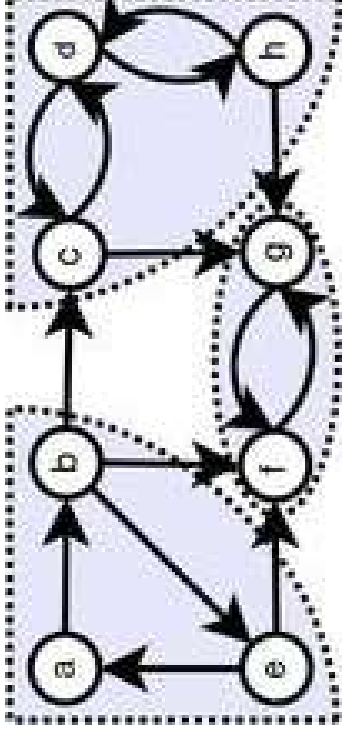
Houda.benaliouche@univ-constantine2.dz

Etudiants concernés

Faculté/Institut	Département	Niveau	Spécialité
Nouvelles technologies	/	Licence 2	/

Prérequis

- Notions mathématiques
- Notions algorithmiques



Objectifs du cours

- Définir la structure d'arbre, d'arborescence et de forêt.
- Définir la problématique d'arbres couvrant de poids min/max
- Appliquer quelques algorithmes d'arbres couvrant de poids min/max.

Section 1 : Définitions

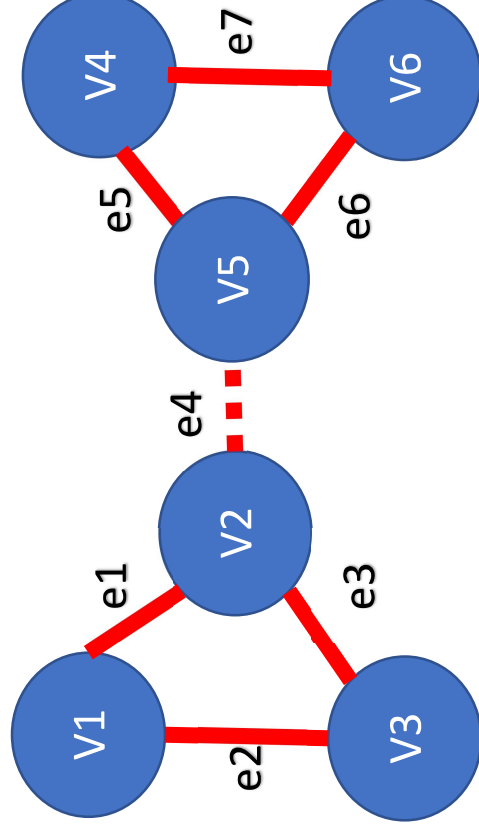
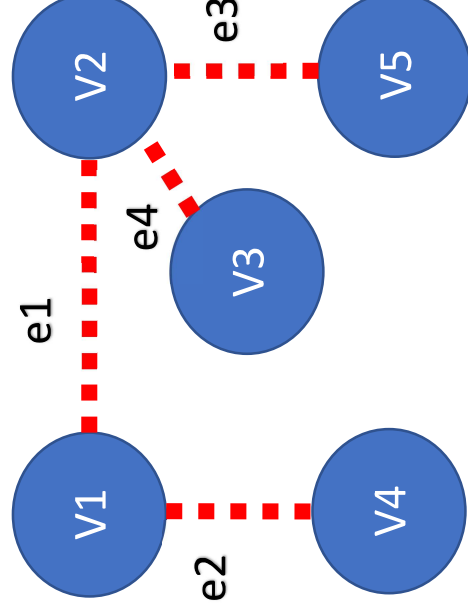
Isthme

- Isthme (*Bridge*)

Une arête est un isthme si sa suppression augmente le nombre de composantes connexes d'un graphe.

Dans le graphe G_1 seule l'arête e_4 est un isthme.

Dans le graphe G2 toutes les arêtes sont des isthmes.

$$G1 = (\{V1, V2, V3, V4, V5, V6\}, \{e1, e2, e3, e4, e5, e6, e7\})$$

$$G2 = (\{V1, V2, V3, V4, V5\}, \{e1, e2, e3, e4\})$$


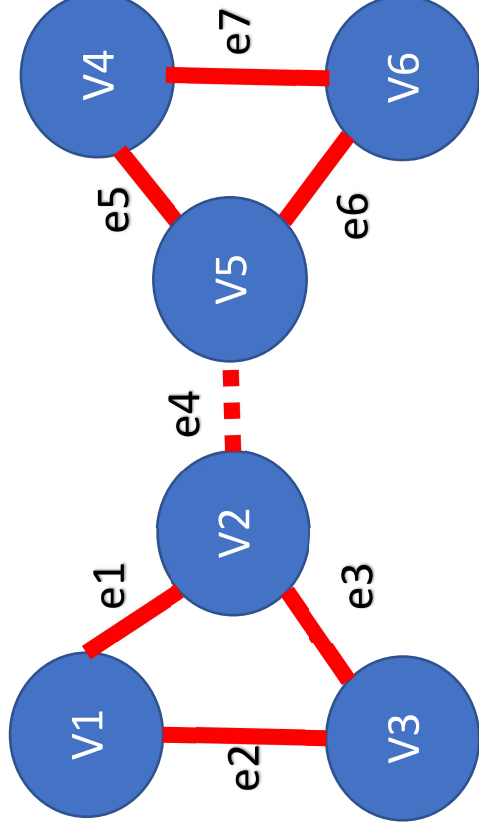
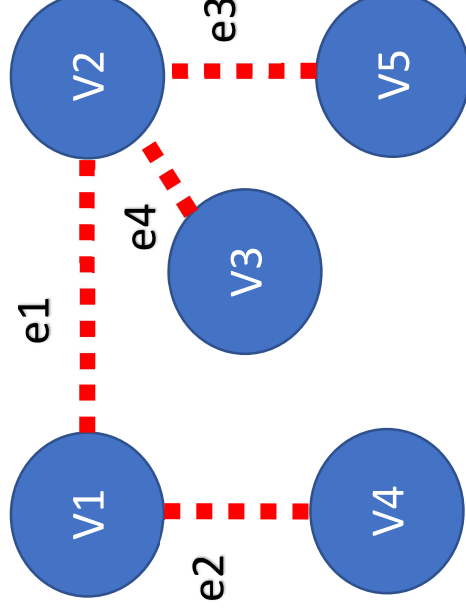
Isthme

- Isthme (*Bridge*)

Une arête d'un graphe est un isthme si elle n'appartient à aucun cycle.

Dans le graphe G1 seule l'arête e4 est un isthme (e4 n'appartient à aucun cycle).

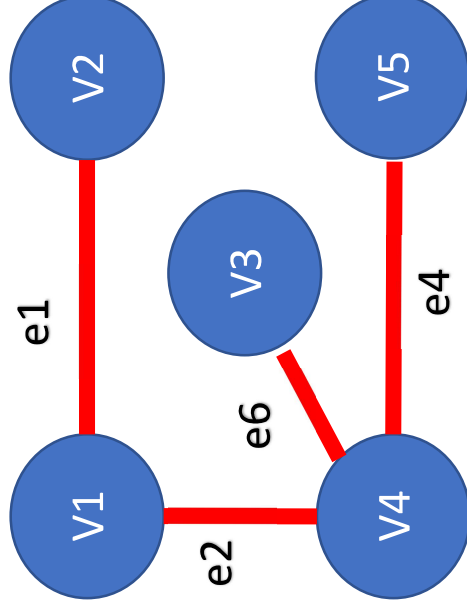
Dans le graphe G_2 toutes les arêtes sont des isthmes (toutes les arêtes du graphe G_2 n'appartiennent à aucun cycle).


$$G2 = (\{V1, V2, V3, V4, V5\}, \{e1, e2, e3, e4\})$$


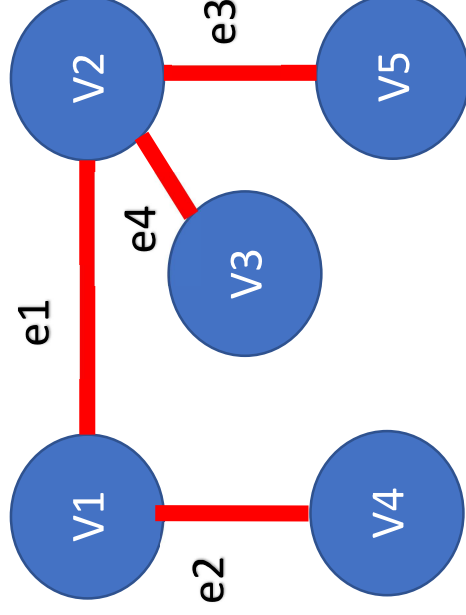
- Arbre (*Tree*)

Un arbre (*Tree en Anglais*) est un graphe connexe ne contenant aucun cycle.
Toutes les arêtes d'un arbre sont des isthmes.

$G1 = (\{V1, V2, V3, V4, V5\}, \{e1, e2, e4, e6\})$ est un arbre



$G2 = (\{V1, V2, V3, V4, V5\}, \{e1, e2, e5, e3\})$ est un arbre



Arbre (cinq propriétés)

- Arbre (**Tree**) propriété 1

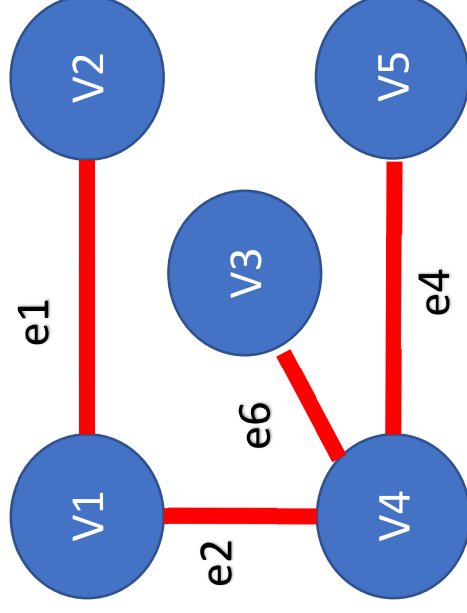
Soit $G=(X,U)$ un graphe avec $|X| = n \geq 2$.

Les propriétés suivantes sont équivalentes et caractérisent un arbre :

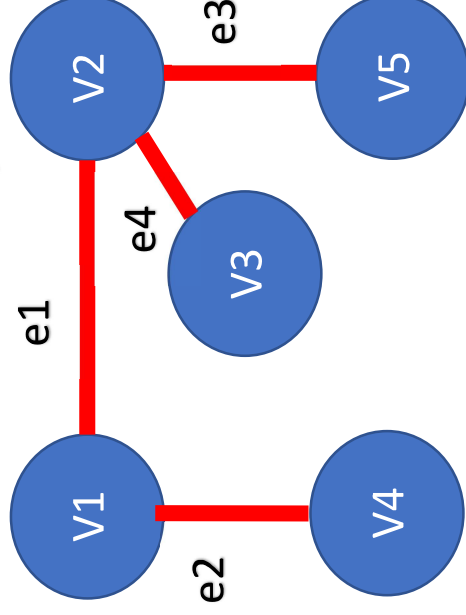
Propriété 1

G est connexe et sans cycle.

$G1=(\{V1,V2,V3,V4,V5\},\{e1,e2,e4,e6\})$ est un arbre connexe et sans cycle



$G2=(\{V1,V2,V3,V4,V5\},\{e1,e2,e5,e3\})$ est un arbre connexe et sans cycle



Arbre (cinq propriétés)

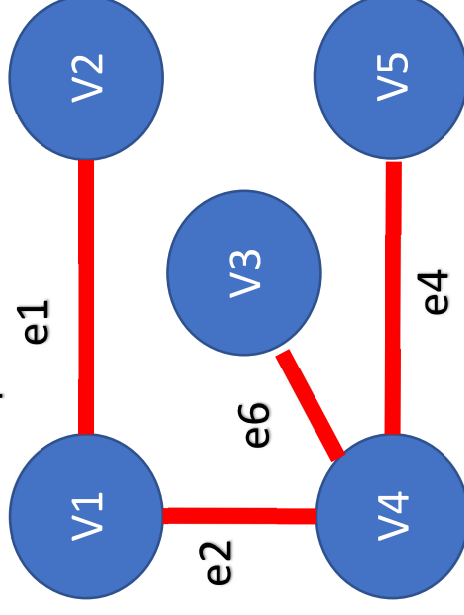
- Arbre (**Tree**) propriété 2

Propriété 2

G est connexe et est minimal pour la propriété 1. (Si on supprime un arc de G, alors G n'est plus connexe).

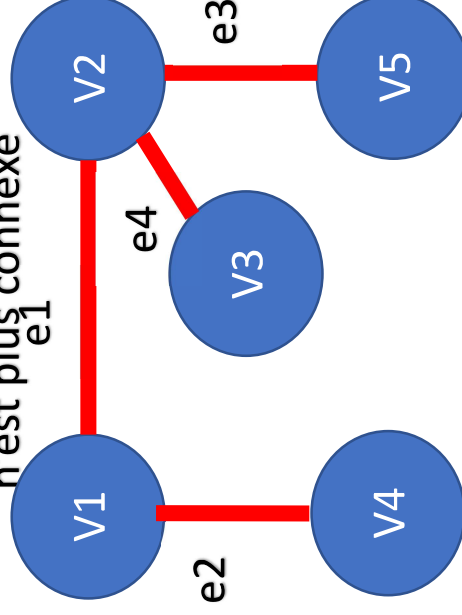
$G1 = (\{V1, V2, V3, V4, V5\}, \{e1, e2, e4, e6\})$ est un arbre connexe et sans cycle, si on supprime une arête il

n'est plus connexe



$G2 = (\{V1, V2, V3, V4, V5\}, \{e1, e2, e5, e3\})$ est un arbre connexe et sans cycle, si on supprime une arête il

n'est plus connexe



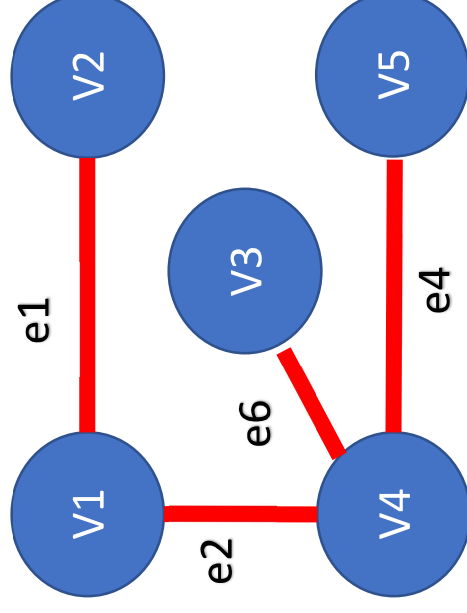
Arbre (cinq propriétés)

- Arbre (**Tree**) propriété 3

Propriété 3

G est connexe et possède $n-1$ arêtes.

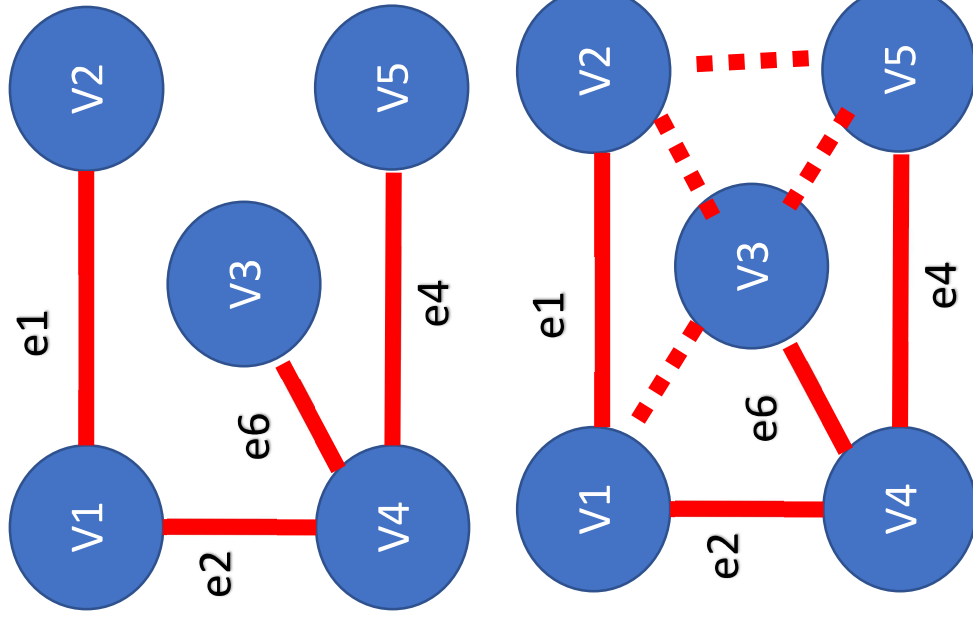
$G1 = (\{V1, V2, V3, V4, V5\}, \{e1, e2, e4, e6\})$ est un arbre connexe et sans cycle, possédant $5-1=4$ arêtes



Arbre (cinq propriétés)

- Arbre (**Tree**) propriété 4

$G1 = (\{V1, V2, V3, V4, V5\}, \{e1, e2, e4, e6\})$ est un arbre connexe et sans cycle, possédant $5-1=4$ arêtes, si on ajoute une arête, on forme un cycle



Propriété 4

**G est sans cycle et est maximal pour la propriété 3.
(Si on ajoute un arc à G , alors il possède un cycle).**

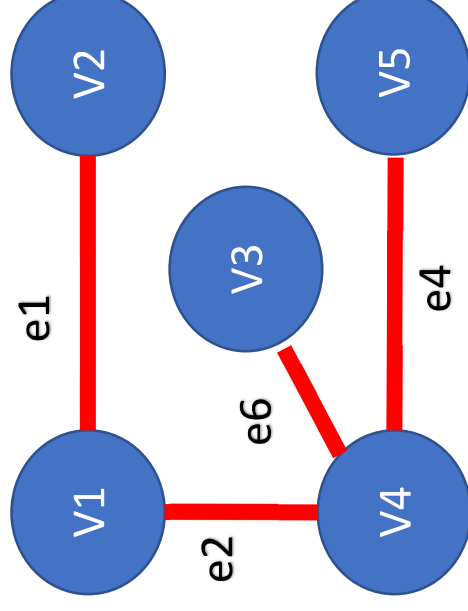
Arbre (cinq propriétés)

- Arbre (**Tree**) propriété 5

$G1 = (\{V1, V2, V3, V4, V5\}, \{e1, e2, e4, e6\})$ est un arbre connexe et sans cycle, possédant $5 - 1 = 4$ arêtes, **$\exists$ dans G une chaîne et une seule joignant tout couple de sommets**

Propriété 5

$\exists$ dans G une chaîne et une seule joignant tout couple de sommets.

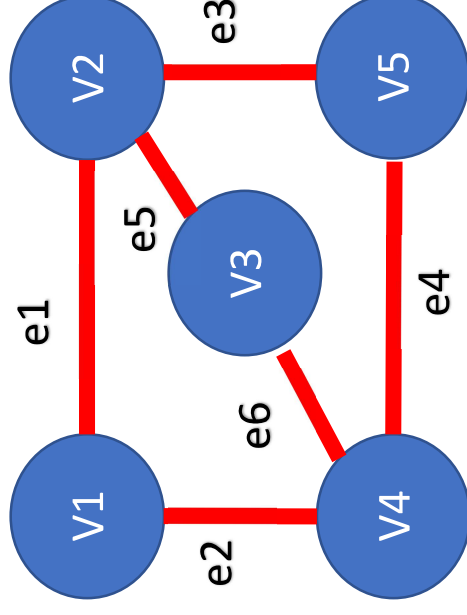


Arbre couvrant

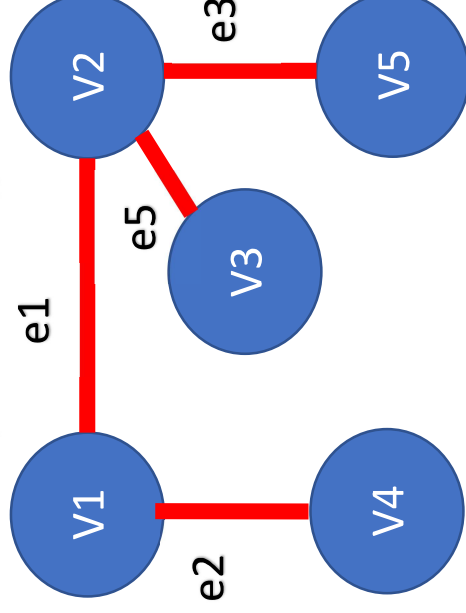
- Arbre couvrant (*Spanning Tree*)

Un arbre couvrant (*Spanning Tree en Anglais*) est un sous graphe maximum d'un graphe ne contenant aucun cycle (qui est aussi un arbre).
Maximum veut dire contenant tous les sommets de l'arbre.

$G = (\{V1, V2, V3, V4, V5\}, \{e1, e2, e3, e4, e5, e6\})$



Arbre couvrant de G arbreG = $(\{V1, V2, V3, V4, V5\}, \{e1, e2, e5, e3\})$



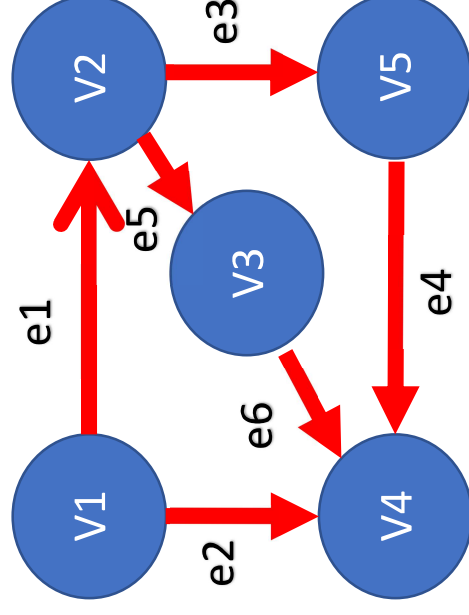
Arborescence

- Arborescence (*Rooted tree*)

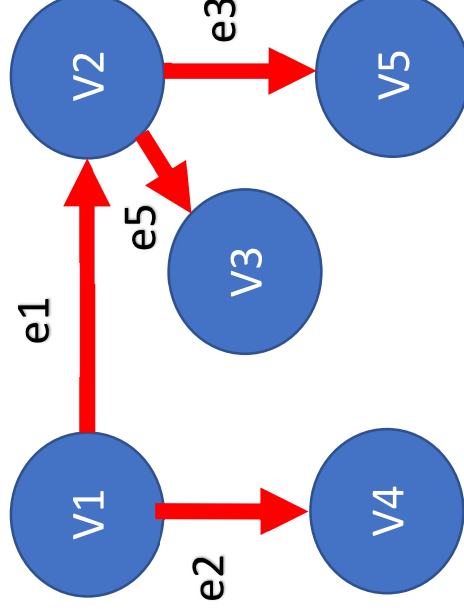
Une arborescence (*Rooted tree en Anglais*) est un arbre avec sommet distingué ou privilégié (*Root*).

exemple: RootedG est une arborescence avec le sommet V1 comme racine, car RootedG est un arbre + il existe un et un seul chemin de V1 vers tous les sommets.

$G = (\{V1, V2, V3, V4, V5\}, \{e1, e2, e3, e4, e5, e6\})$



Arborescence RootedG = $(\{V1, V2, V3, V4, V5\}, \{e1, e2, e5, e3\})$



- Forêt (*Forest*)

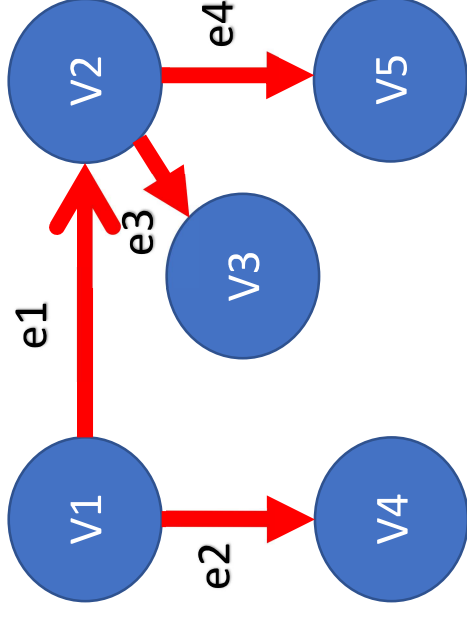
Une forêt (*Forest en Anglais*) est un ensemble d'arbres.

Une forêt est un graphe ne contenant pas de cycle.

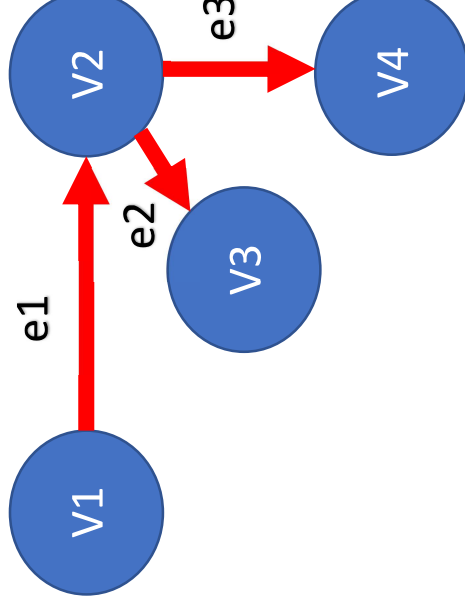
Les composantes connexes de la forêt sont des arbres.

exemple: Arbre 1 et Arbre2 sont deux composantes d'une forêt.

$\text{Arbre1} = (\{V1, V2, V3, V4, V5\}, \{e1, e2, e3, e4\})$



$\text{Arbre2} = (\{V1, V2, V3, V4\}, \{e1, e2, e3\})$



Section 2:

Problématique d'arbres couvrant de poids min/max

La problématique se pose comme suit : étant donné un graphe avec un nombre de sommets et un nombre d'arêtes ayant des poids de valeurs dans l'ensemble des entiers relatifs

- l'arbre couvrant minimal consiste à trouver l'ensemble des arêtes permettant de rejoindre tous les sommets sans former de cycle, et ce, avec un coût minimal

graphe → Spanning tree
arbre avec tous les sommets et coût minimal

- Ce problème trouve des applications pratiques variées :
il est directement applicable pour
- l'optimisation
 - la conception de divers types de réseaux (électrique, internet, etc.).

Section 3: application d'algorithmes (Kruskal, Prim, Sollin)

Histoire des algorithmes d'arbres couvrant

- Algorithme de Kruskal (1956)
- Algorithme de Prim (1957)
 - Prim-dijkstra, jarnik (1930)
- Algorithme de Boruvka (1926)
 - Sollin (1961)

L'histoire des algorithmes d'arbres couvrants de poids minimum est à l'image de l'histoire de la recherche en algorithmique.

Les algorithmes d'arbres couvrants de poids minimal les plus connus sont ceux de Kruskal et de Prim

...

Principe de l'algorithme de Kruskal

- Trier les poids des arêtes dans l'ordre croissant.
- Ajouter les arêtes et leurs sommets à l'arbre couvrant si pas de création de cycle.
- S'arrêter si le nombre d'arêtes de l'arbre couvrant atteint le nombre de sommets de l'arbre d'origine moins 1.

Kruskal maintient un arbre acyclique à chaque rajout d'arête

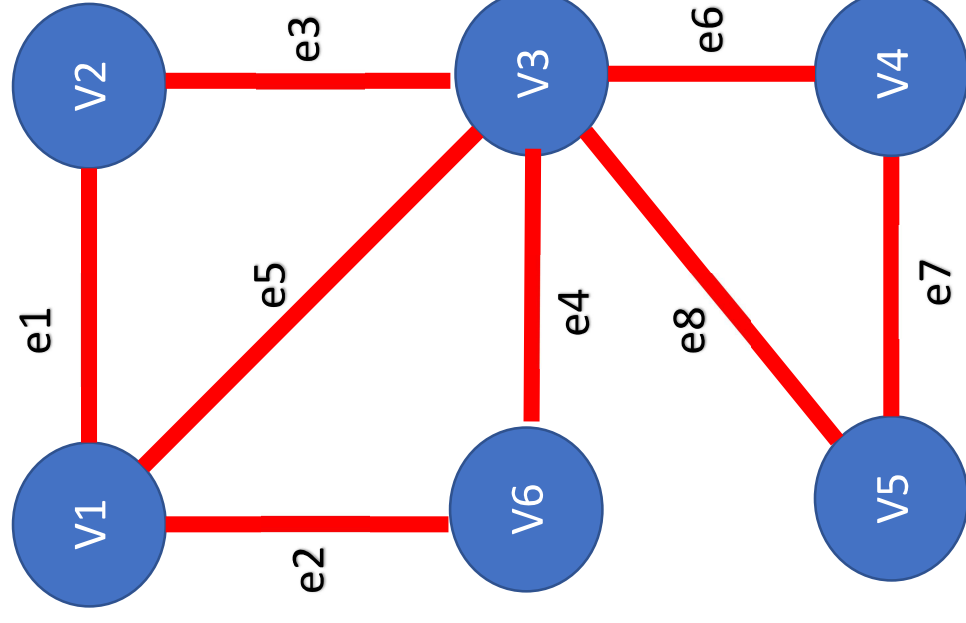
...

Algorithme de Kruskal

- Construire un arbre couvrant de poids minimal

Les données du problème:

Un graphe connexe et pondéré $G = (V, E)$ défini par l'ensemble fini de sommets $V = \{v_1, v_2, \dots, v_n\}$ et par l'ensemble fini des arêtes valuées $E = \{e_1, e_2, \dots, e_m\}$

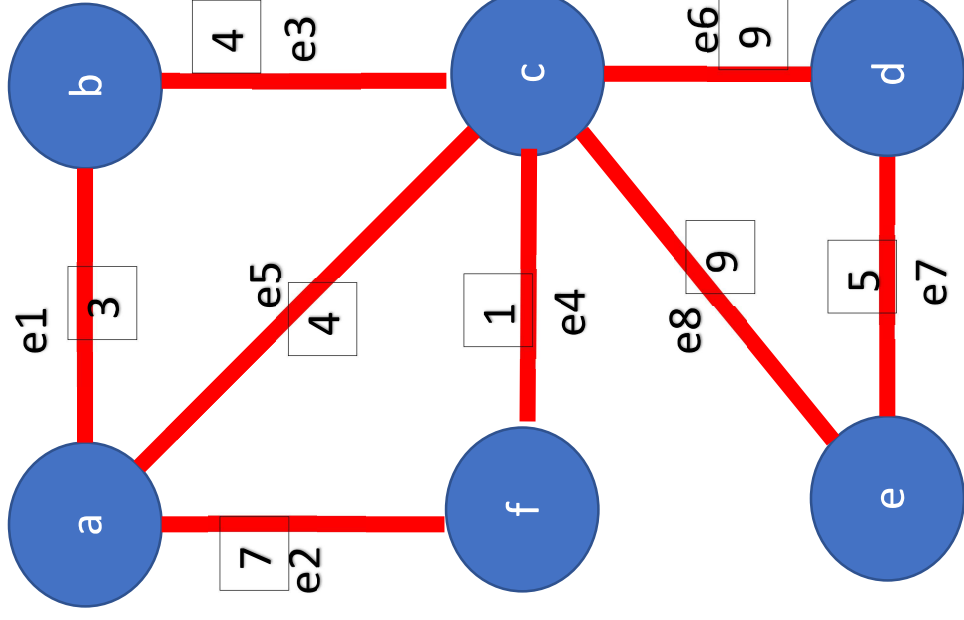


Algorithme de Kruskal

- Construire un arbre couvrant de poids minimal

Exemple:

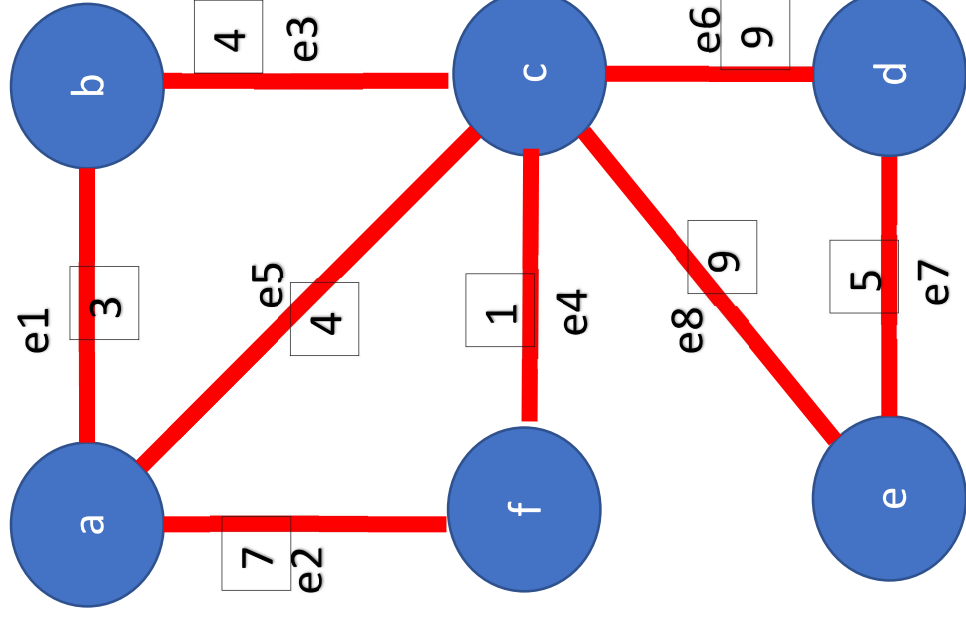
Un graphe connexe et pondéré $G = (V, E)$ défini par l'ensemble fini de sommets $V = \{a, b, c, d, e, f\}$ et par l'ensemble fini des arêtes valuées $E = \{e_1, e_2, e_3, e_4, e_5, e_6, e_7\}$ avec les poids respectives $\{3, 7, 4, 1, 4, 9, 5\}$



Algorithme de Kruskal (première exécution)

- Construire un arbre couvrant de poids minimal
 - Étape 1: Trier les arêtes par poids croissant

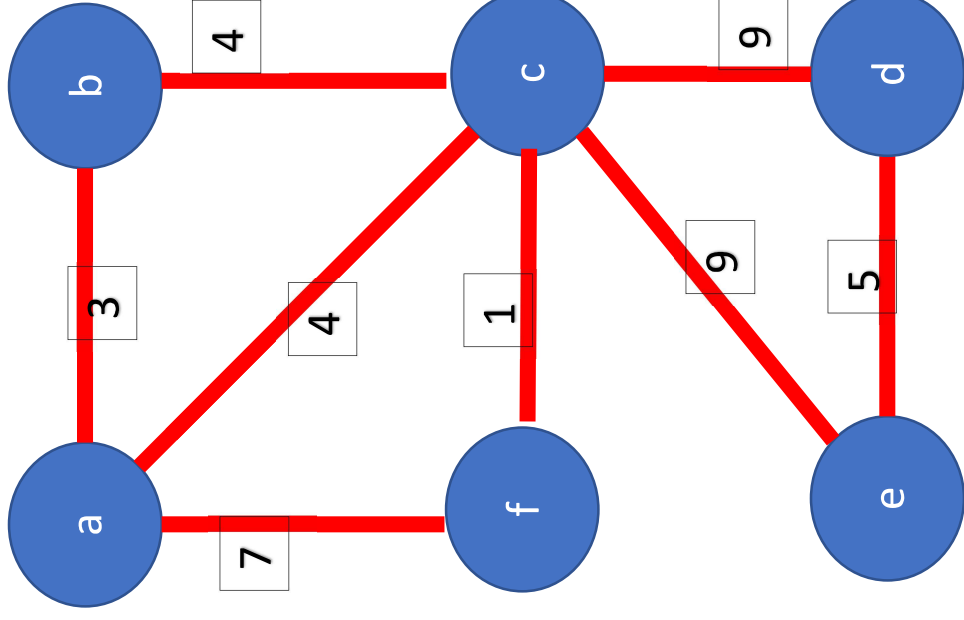
f,c: 1
a,b: 3
b,c: 4 **première exécution**
a,c: 4
e,d: 5
a,f: 7
c,d: 9
c,e: 9



Algorithme de Kruskal (première exécution)

- Construire un arbre couvrant de poids minimal
 - Étape 1: Trier les arêtes par poids croissant

f,c: 1
a,b: 3
b,c: 4
a,c: 4
e,d: 5
a,f: 7
c,d: 9
c,e: 9



Algorithme de Kruskal (première exécution)

- Construire un arbre couvrant de poids minimal

- Étape 2: Ajouter les arêtes une à une si pas de création de cycle

f,c : 1

a,b : 3

b,c : 4

a,c : 4

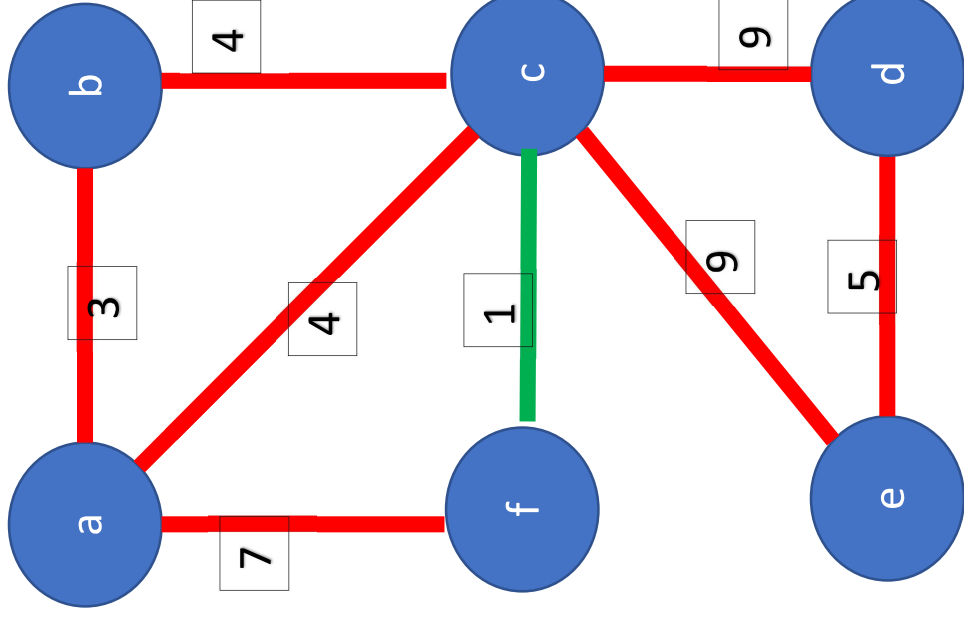
e,d : 5

a,f : 7

c,d : 9

c,e : 9

f,c : 1 ✓



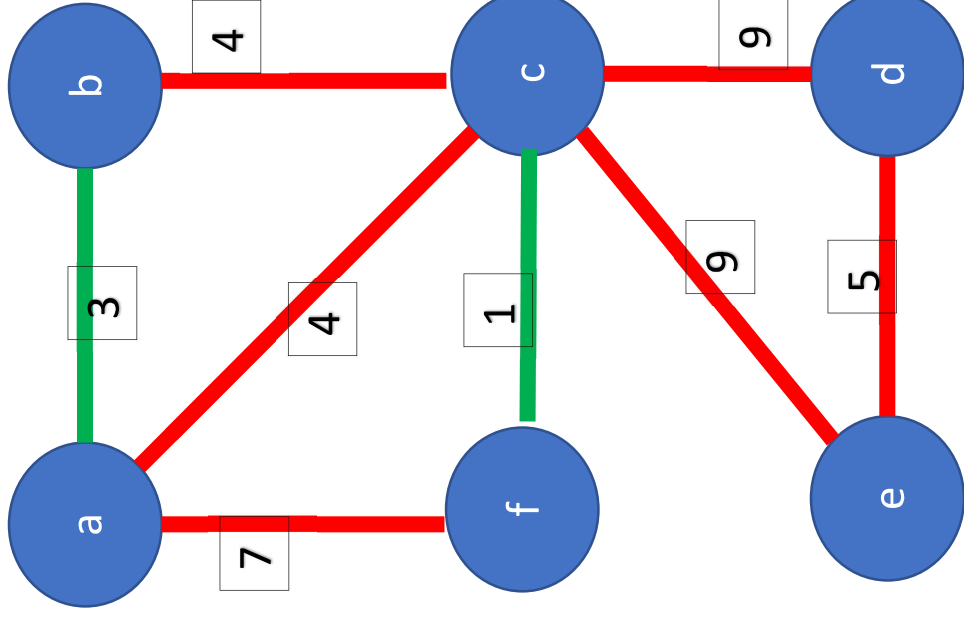
Algorithme de Kruskal (première exécution)

- Construire un arbre couvrant de poids minimal

- Étape 2: Ajouter les arêtes une à une si pas de création de cycle

f,c: 1
a,b: 3
b,c: 4
a,c: 4
e,d: 5
a,f: 7
c,d: 9
c,e: 9

f,c: 1
a,b: 3



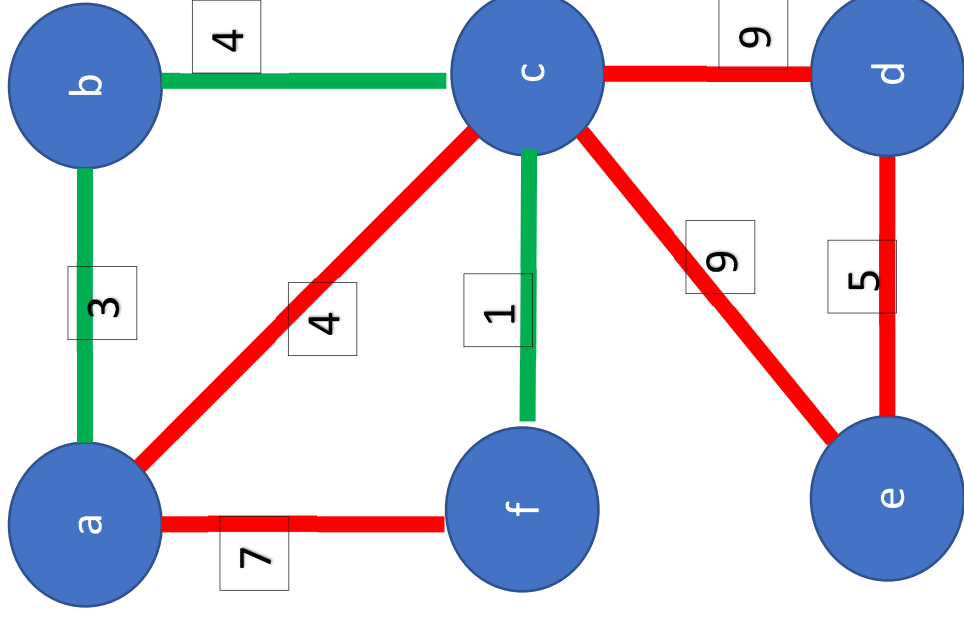
Algorithme de Kruskal (première exécution)

- Construire un arbre couvrant de poids minimal

- Étape 2: Ajouter les arêtes une à une si pas de création de cycle

f,c: 1
a,b: 3
b,c: 4
a,c: 4
e,d: 5
a,f: 7
c,d: 9
c,e: 9

f,c: 1 ✓
a,b: 3 ✓
b,c: 4 ✓



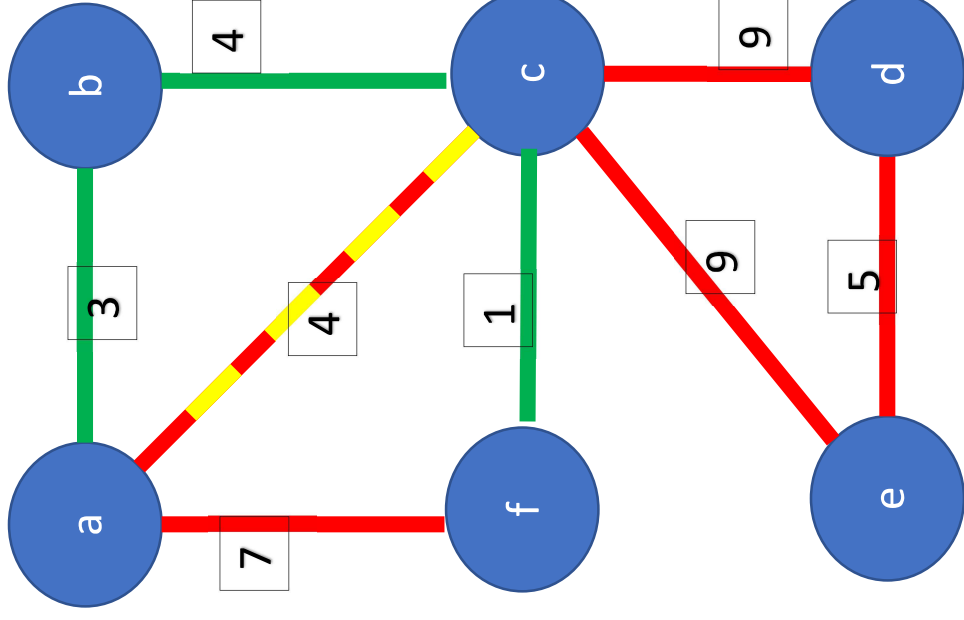
Algorithme de Kruskal (première exécution)

- Construire un arbre couvrant de poids minimal

- Étape 2: Ajouter les arêtes une à une si pas de création de cycle

f,c: 1
a,b: 3
b,c: 4
a,c: 4
e,d: 5
a,f: 7
c,d: 9
c,e: 9

f,c: 1 ✓
a,b: 3 ✓
b,c: 4 ✓
a,c: 4 ✗



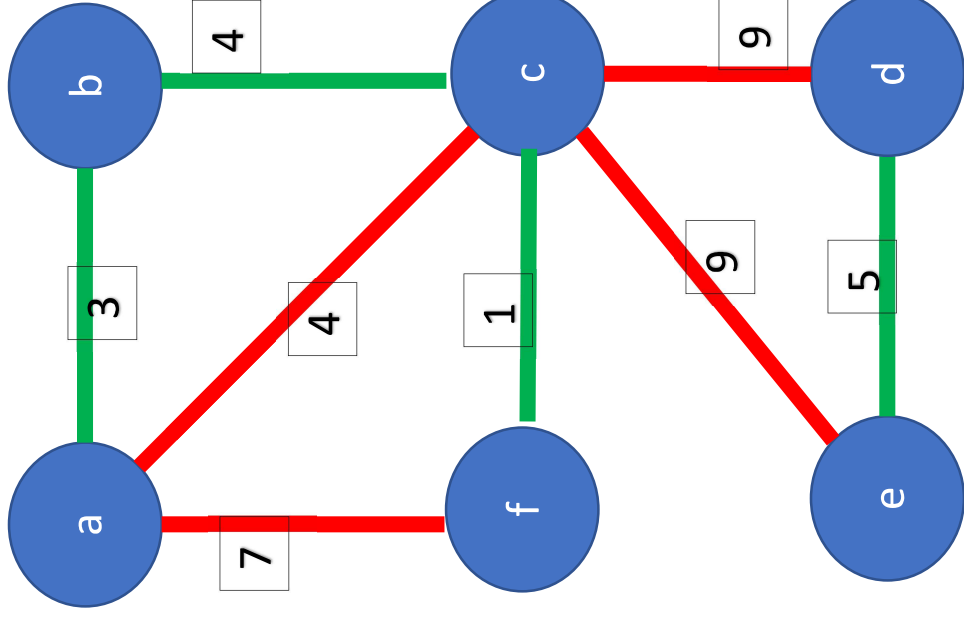
Algorithme de Kruskal (première exécution)

- Construire un arbre couvrant de poids minimal

- Étape 2: Ajouter les arêtes une à une si pas de création de cycle

f,c: 1
a,b: 3
b,c: 4
a,c: 4
e,d: 5
a,f: 7
c,d: 9
c,e: 9

f,c: 1
a,b: 3
b,c: 4
a,c: 4
e,d: 5



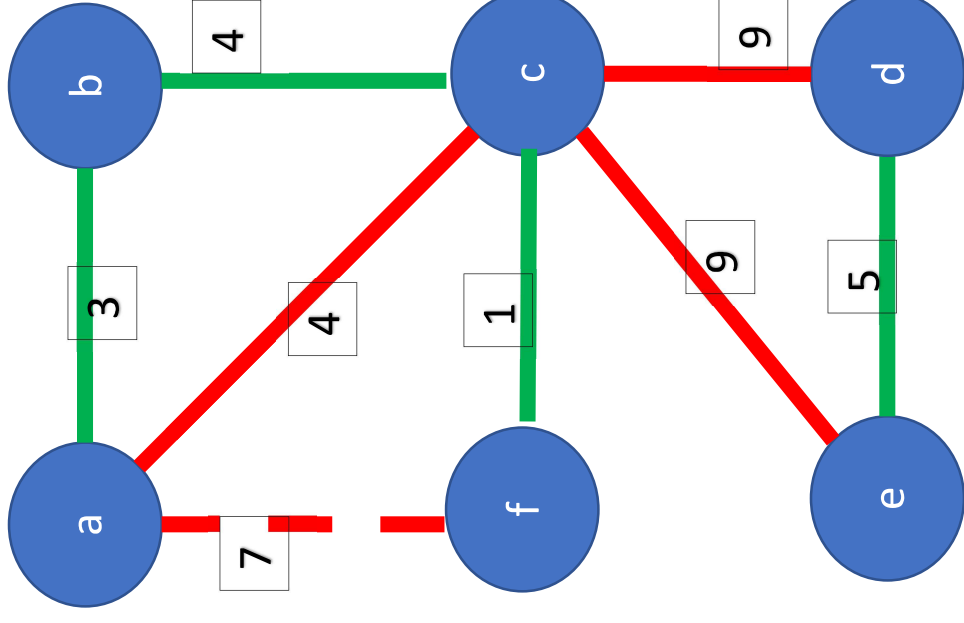
Algorithme de Kruskal (première exécution)

- Construire un arbre couvrant de poids minimal

- Étape 2: Ajouter les arêtes une à une si pas de création de cycle

f,c: 1
a,b: 3
b,c: 4
a,c: 4
e,d: 5
a,f: 7
c,d: 9
c,e: 9

f,c: 1 ✓
a,b: 3 ✓
b,c: 4 ✓
a,c: 4 ✗
e,d: 5 ✓
a,f: 7 ✗



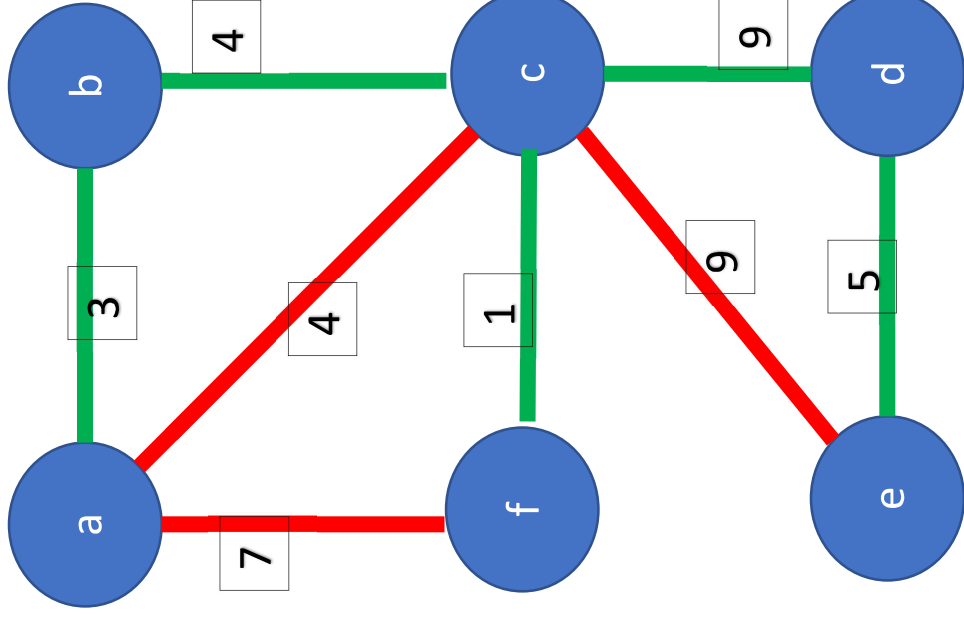
Algorithme de Kruskal (première exécution)

- Construire un arbre couvrant de poids minimal

- Étape 2: Ajouter les arêtes une à une si pas de création de cycle

f,c: 1
a,b: 3
b,c: 4
a,c: 4
e,d: 5
a,f: 7
c,d: 9
c,e: 9

f,c: 1 ✓
a,b: 3 ✓
b,c: 4 ✓
a,c: 4 ✗
e,d: 5 ✓
a,f: 7 ✗
c,d: 9 ✓



Algorithme de Kruskal (première exécution)

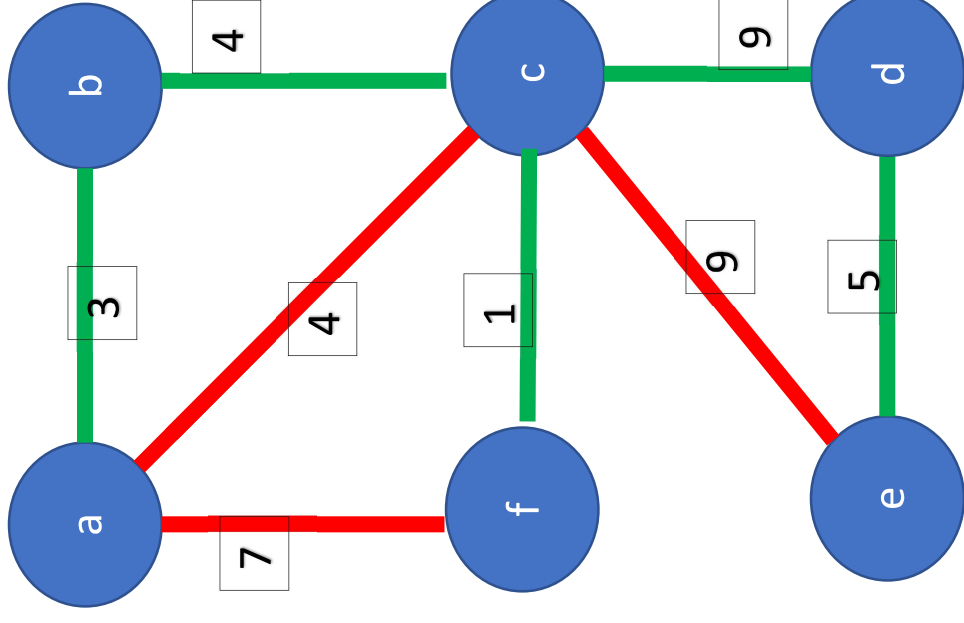
- Construire un arbre couvrant de poids minimal

- Étape 2: Ajouter les arêtes une à une si pas de création de cycle

f,c: 1
a,b: 3
b,c: 4
a,c: 4
e,d: 5
a,f: 7
c,d: 9
c,e: 9

f,c: 1
a,b: 3
b,c: 4
a,c: 4
e,d: 5
a,f: 7
c,d: 9

On s'arrête car l'arbre obtenu couvre tous les sommets



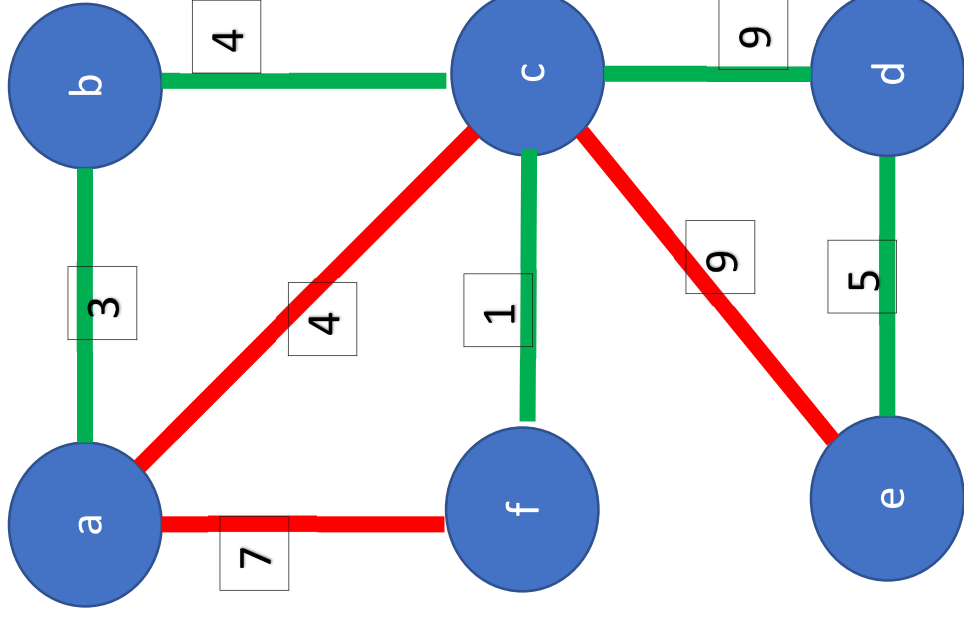
Algorithme de Kruskal (première exécution)

- Construire un arbre couvrant de poids minimal

- Étape 2: Ajouter les arêtes une à une si pas de création de cycle

f,c: 1
a,b: 3
b,c: 4
a,c: 4
e,d: 5
a,f: 7
c,d: 9
c,e: 9

f,c: 1
a,b: 3
b,c: 4
a,c: 4
e,d: 5
a,f: 7
c,d: 9
Poids de cet arbre=
1+3+4+5+9=22



Algorithme de Kruskal (première exécution)

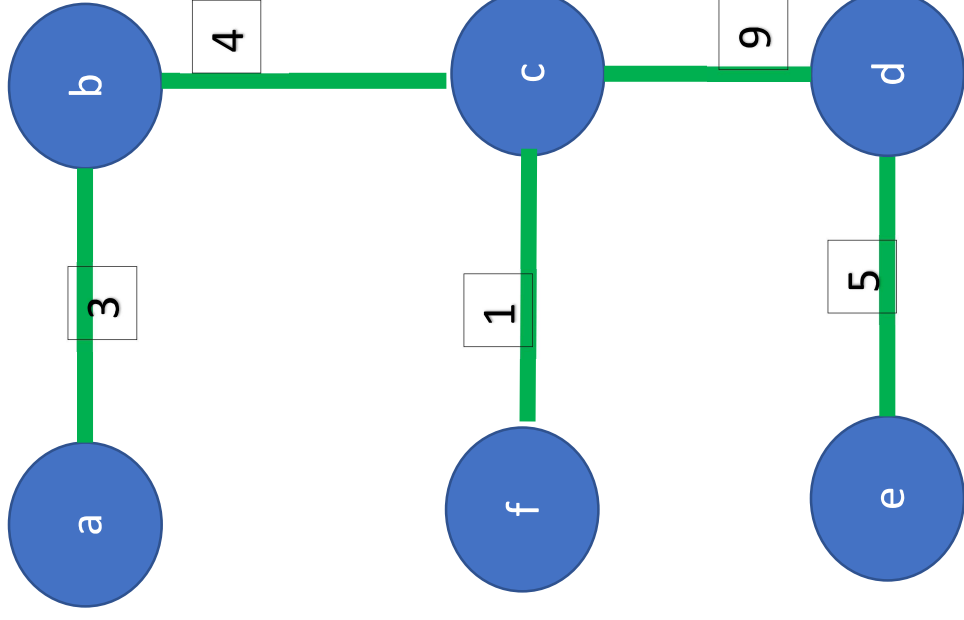
- Construire un arbre couvrant de poids minimal
 - Étape 2: Ajouter les arêtes une à une si pas de création de cycle

f,c: 1
a,b: 3
b,c: 4
a,c: 4
e,d: 5
a,f: 7
c,d: 9
c,e: 9

f,c: 1
a,b: 3
b,c: 4
a,c: 4
e,d: 5
a,f: 7
c,d: 9
Poids de cet arbre=
 $1+3+4+5+9=22$



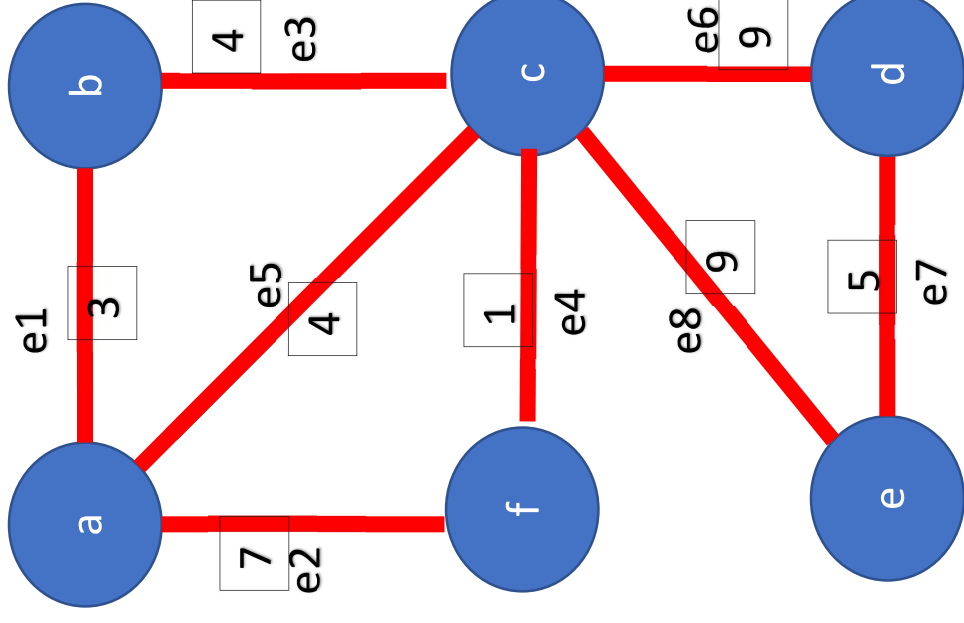
Graphe connexe sans cycle contenant les six sommets d'origine



Algorithme de Kruskal (deuxième exécution)

- Construire un arbre couvrant de poids minimal
 - Étape 1: Trier les arêtes par poids croissant

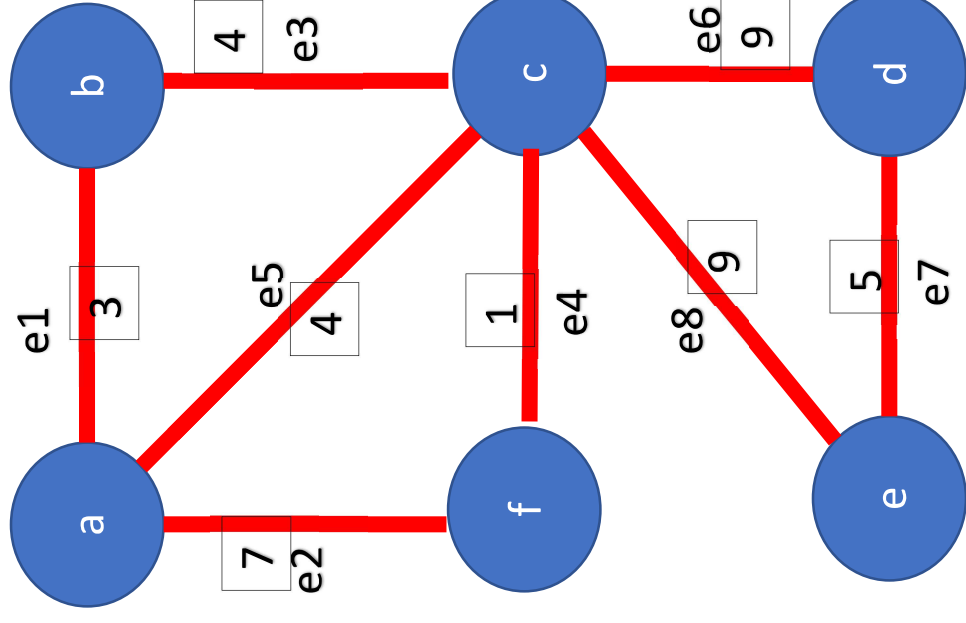
f,c: 1
a,b: 3
b,c: 4 **deuxième exécution**
a,c: 4
e,d: 5
a,f: 7
c,d: 9 **inverser l'ordre**
c,e: 9



Algorithme de Kruskal (deuxième exécution)

- Construire un arbre couvrant de poids minimal
 - Étape 1: Trier les arêtes par poids croissant

f,c: 1
a,b: 3
a,c: 4
b,c: 4
e,d: 5
a,f: 7
c,e: 9
c,d: 9



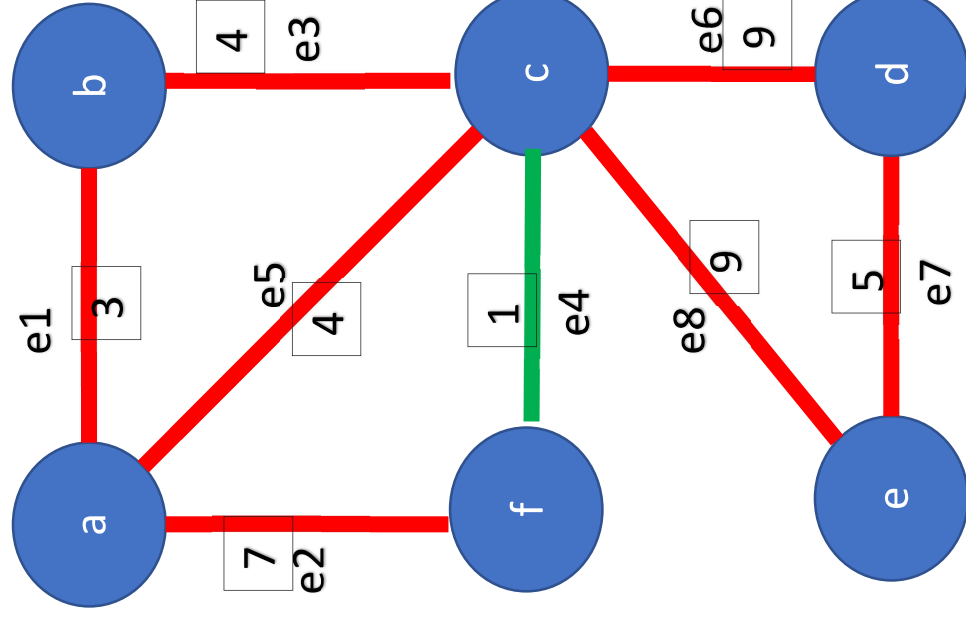
Algorithme de Kruskal (deuxième exécution)

- Construire un arbre couvrant de poids minimal

- Étape 2: Ajouter les arêtes une à une si pas de création de cycle

f,c: 1
a,b: 3
a,c: 4
b,c: 4
e,d: 5
a,f: 7
c,e: 9
c,d: 9

f,c: 1
a,b: 3
a,c: 4
b,c: 4
e,d: 5
a,f: 7
c,e: 9



Algorithme de Kruskal (deuxième exécution)

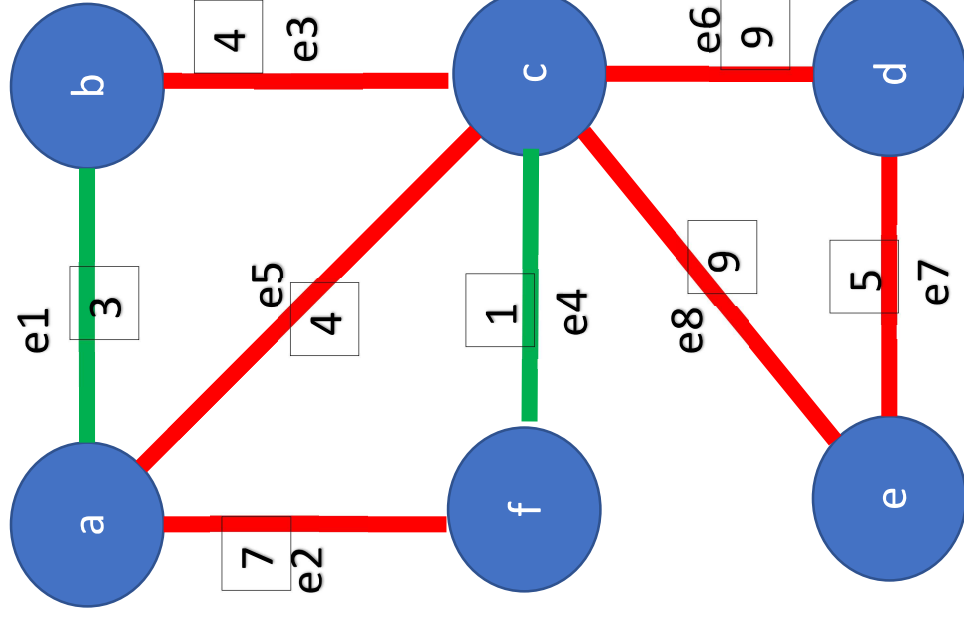
- Construire un arbre couvrant de poids minimal

- Étape 2: Ajouter les arêtes une à une si pas de création de cycle

f,c: 1
a,b: 3
a,c: 4
b,c: 4
e,d: 5
a,f: 7
c,e: 9
c,d: 9



f,c: 1
a,b: 3
a,c: 4
b,c: 4
e,d: 5
a,f: 7
c,e: 9



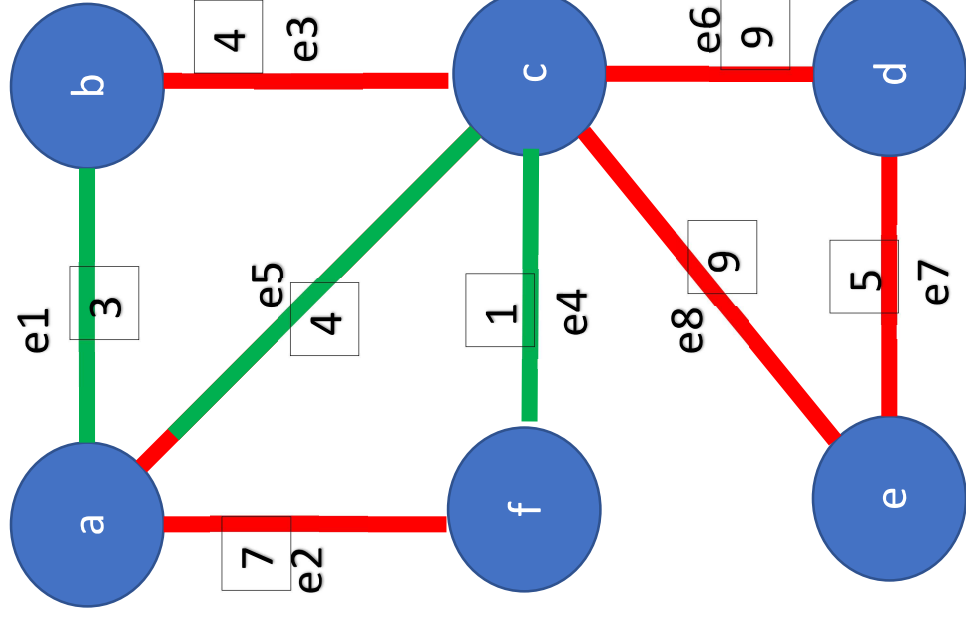
Algorithme de Kruskal (deuxième exécution)

- Construire un arbre couvrant de poids minimal

- Étape 2: Ajouter les arêtes une à une si pas de création de cycle

f,c: 1
a,b: 3
a,c: 4
b,c: 4
e,d: 5
a,f: 7
c,e: 9
c,d: 9

f,c: 1
a,b: 3
a,c: 4
b,c: 4
e,d: 5
a,f: 7
c,e: 9



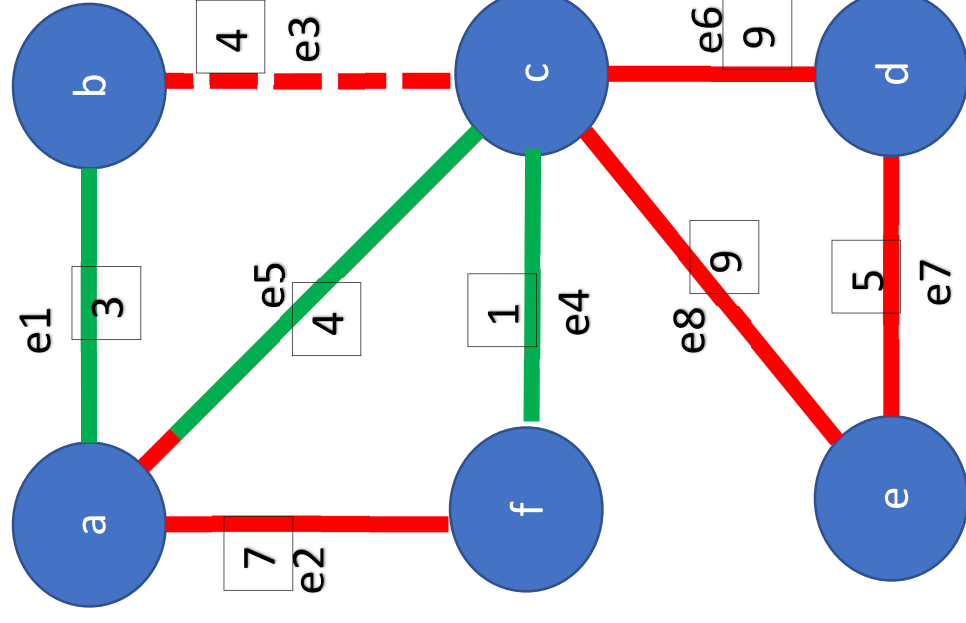
Algorithme de Kruskal (deuxième exécution)

- Construire un arbre couvrant de poids minimal

- Étape 2: Ajouter les arêtes une à une si pas de création de cycle

f,c: 1
a,b: 3
a,c: 4
b,c: 4
e,d: 5
a,f: 7
c,e: 9
c,d: 9

f,c: 1
a,b: 3
a,c: 4
b,c: 4
e,d: 5
a,f: 7
c,e: 9



Algorithme de Kruskal (deuxième exécution)

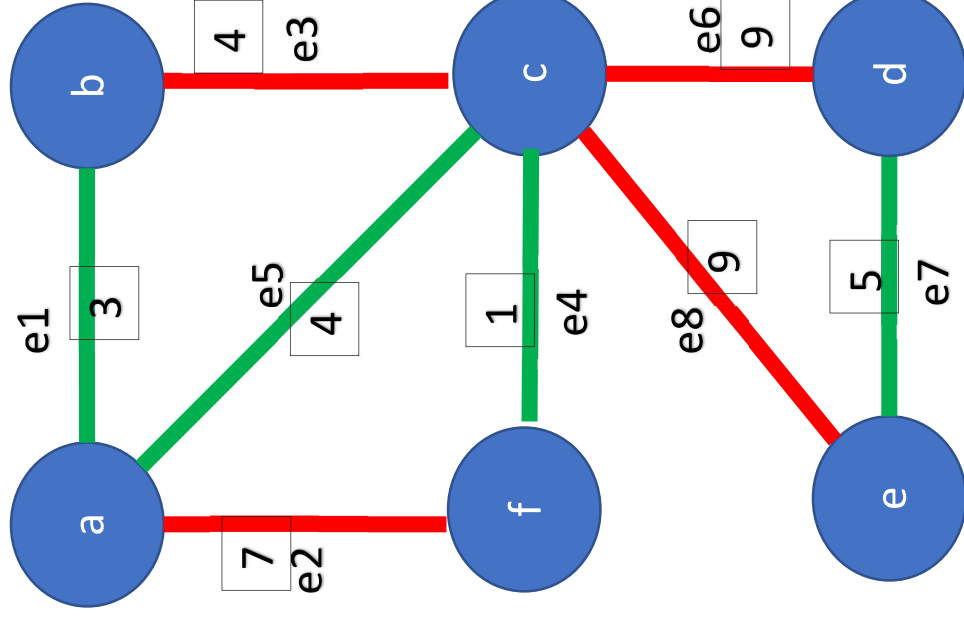
- Construire un arbre couvrant de poids minimal

- Étape 2: Ajouter les arêtes une à une si pas de création de cycle

f,c: 1
a,b: 3
a,c: 4
b,c: 4
e,d: 5
a,f: 7
c,e: 9
c,d: 9

f,c: 1
a,b: 3
a,c: 4
b,c: 4
e,d: 5
a,f: 7
c,e: 9

✓ ✓ ✓ ✗ ✓



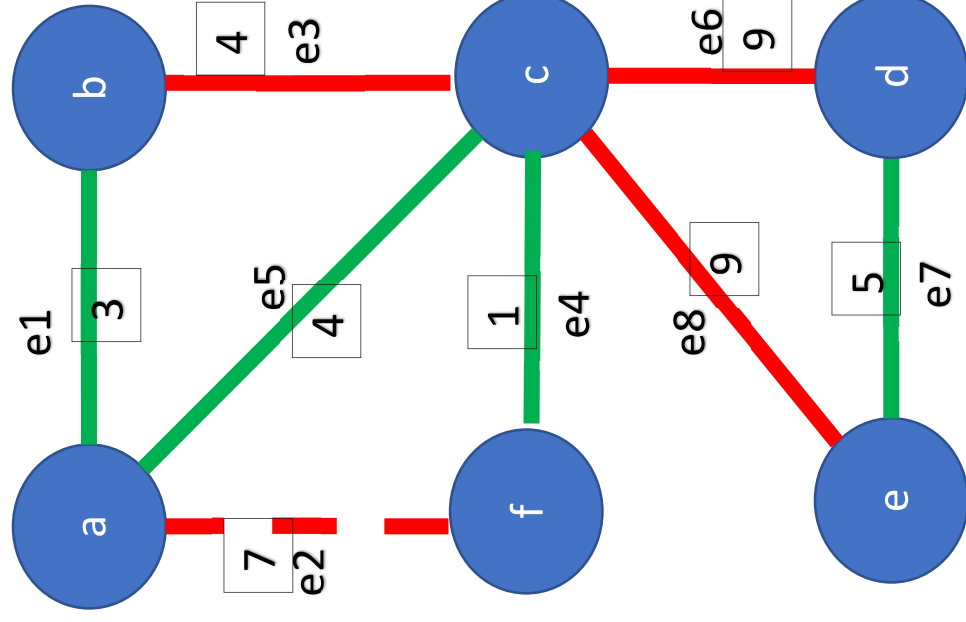
Algorithme de Kruskal (deuxième exécution)

- Construire un arbre couvrant de poids minimal

- Étape 2: Ajouter les arêtes une à une si pas de création de cycle

f,c: 1
a,b: 3
a,c: 4
b,c: 4
e,d: 5
a,f: 7
c,e: 9
c,d: 9

f,c: 1
a,b: 3
a,c: 4
b,c: 4
e,d: 5
a,f: 7
c,e: 9



Algorithme de Kruskal (deuxième exécution)

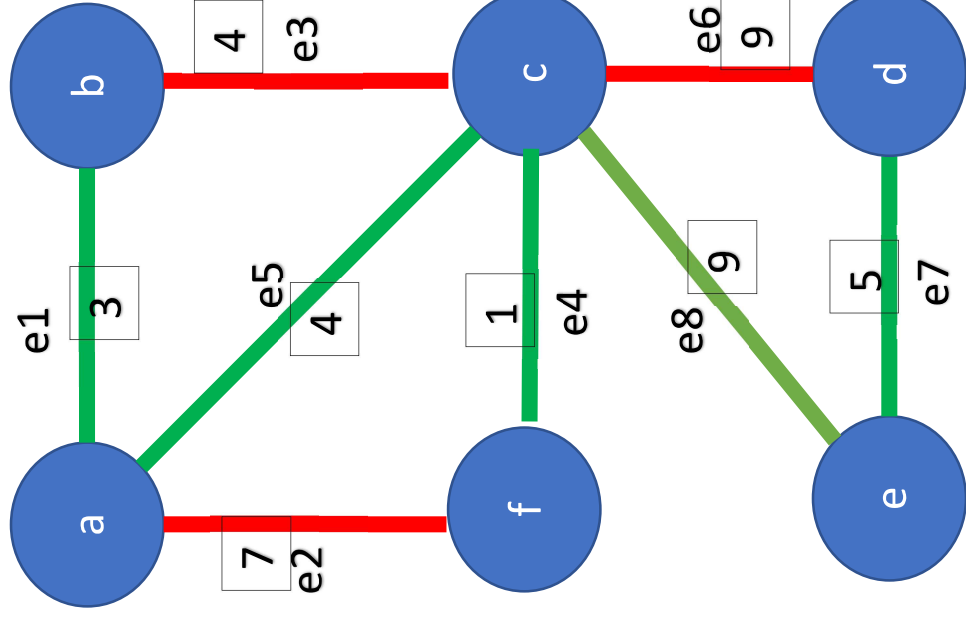
- Construire un arbre couvrant de poids minimal

- Étape 2: Ajouter les arêtes une à une si pas de création de cycle

f,c: 1
a,b: 3
a,c: 4
b,c: 4
e,d: 5
a,f: 7
c,e: 9
c,d: 9

f,c: 1
a,b: 3
a,c: 4
b,c: 4
e,d: 5
a,f: 7
c,e: 9

On s'arrête car l'arbre couvre tous les sommets



Algorithme de Kruskal (deuxième exécution)

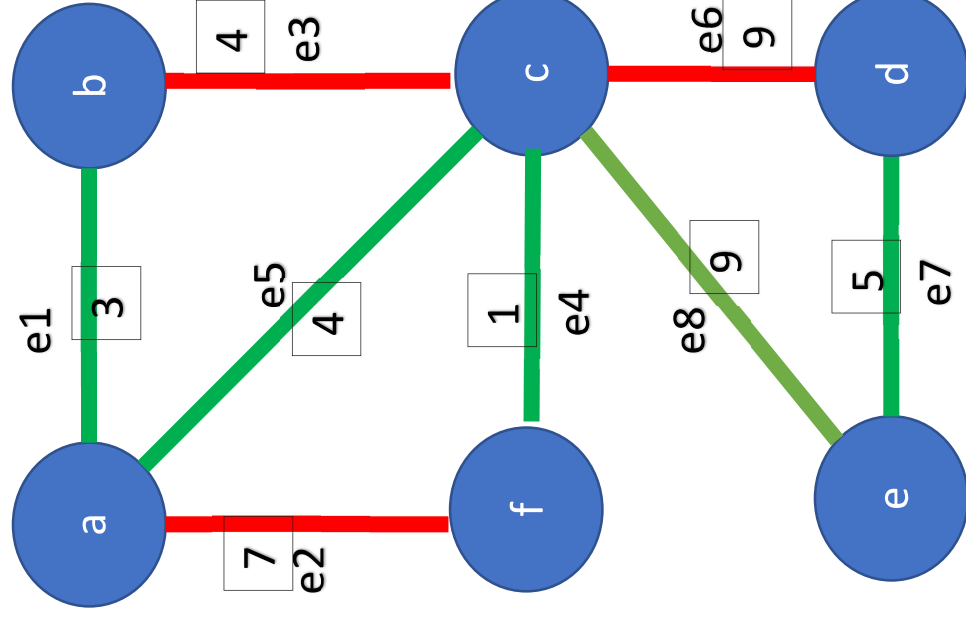
- Construire un arbre couvrant de poids minimal

- Étape 2: Ajouter les arêtes une à une si pas de création de cycle

f,c: 1
a,b: 3
a,c: 4
b,c: 4
e,d: 5
a,f: 7
c,e: 9
c,d: 9

f,c: 1
a,b: 3
a,c: 4
b,c: 4
e,d: 5
a,f: 7
c,e: 9

Poids de l'arbre couvrant
 $= 1+3+4+5+9 = 22$



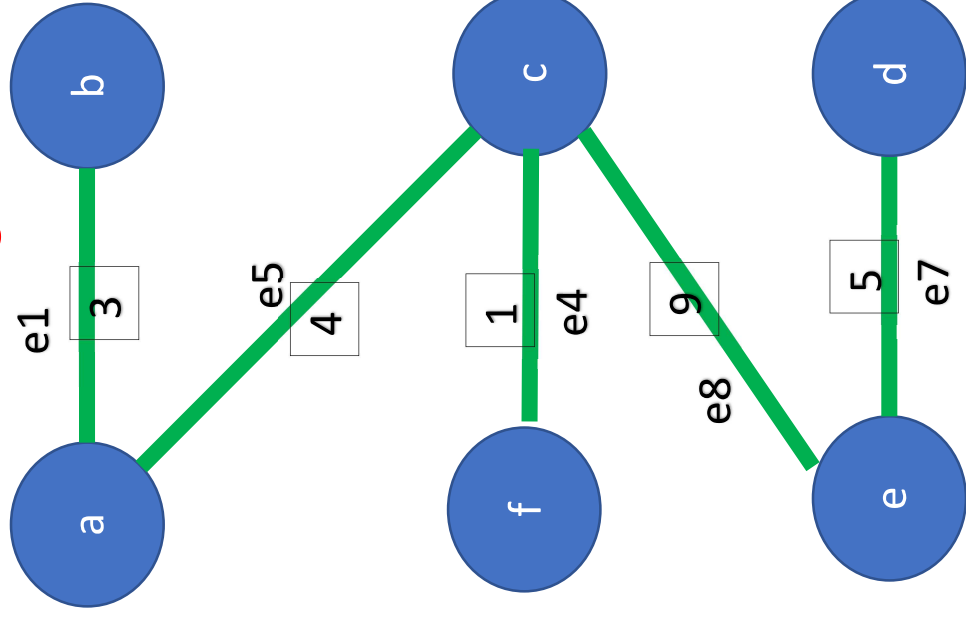
Algorithme de Kruskal (deuxième exécution)

- Construire un arbre couvrant de poids minimal
 - Étape 2: Ajouter les arêtes une à une si pas de création de cycle

f,c: 1
a,b: 3
a,c: 4
b,c: 4
e,d: 5
a,f: 7
c,e: 9
c,d: 9

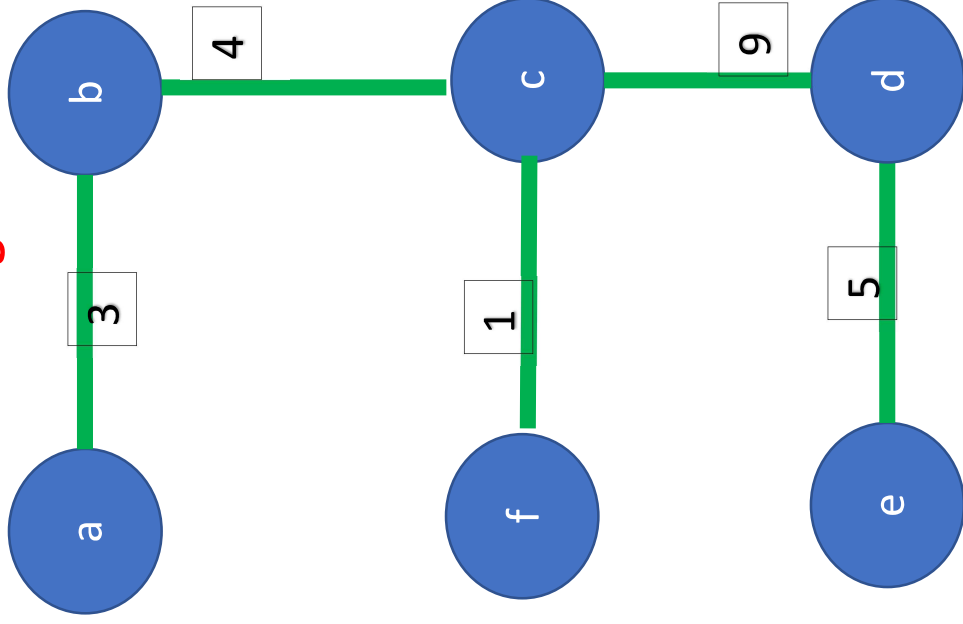
Poids de l'arbre couvrant
=1+3+4+5+9=22

Graphe connexe sans cycle contenant les six sommets d'origine



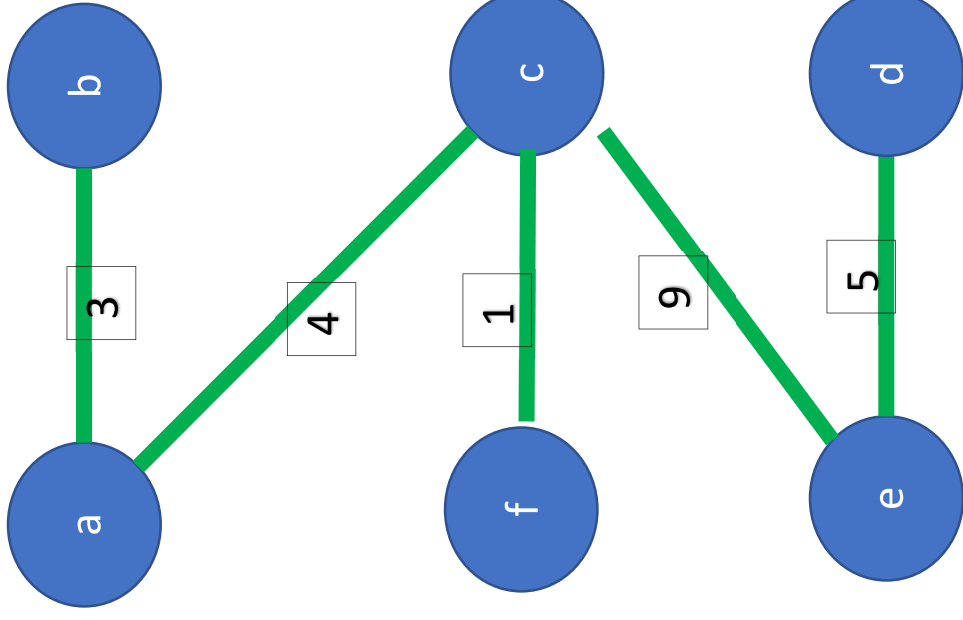
Algorithme de Kruskal (comparer les deux exécutions)

Graphe connexe sans cycle contenant les six sommets d'origine



Poids de l'arbre couvrant = $1+3+4+5+9=22$

Graphe connexe sans cycle contenant les six sommets d'origine

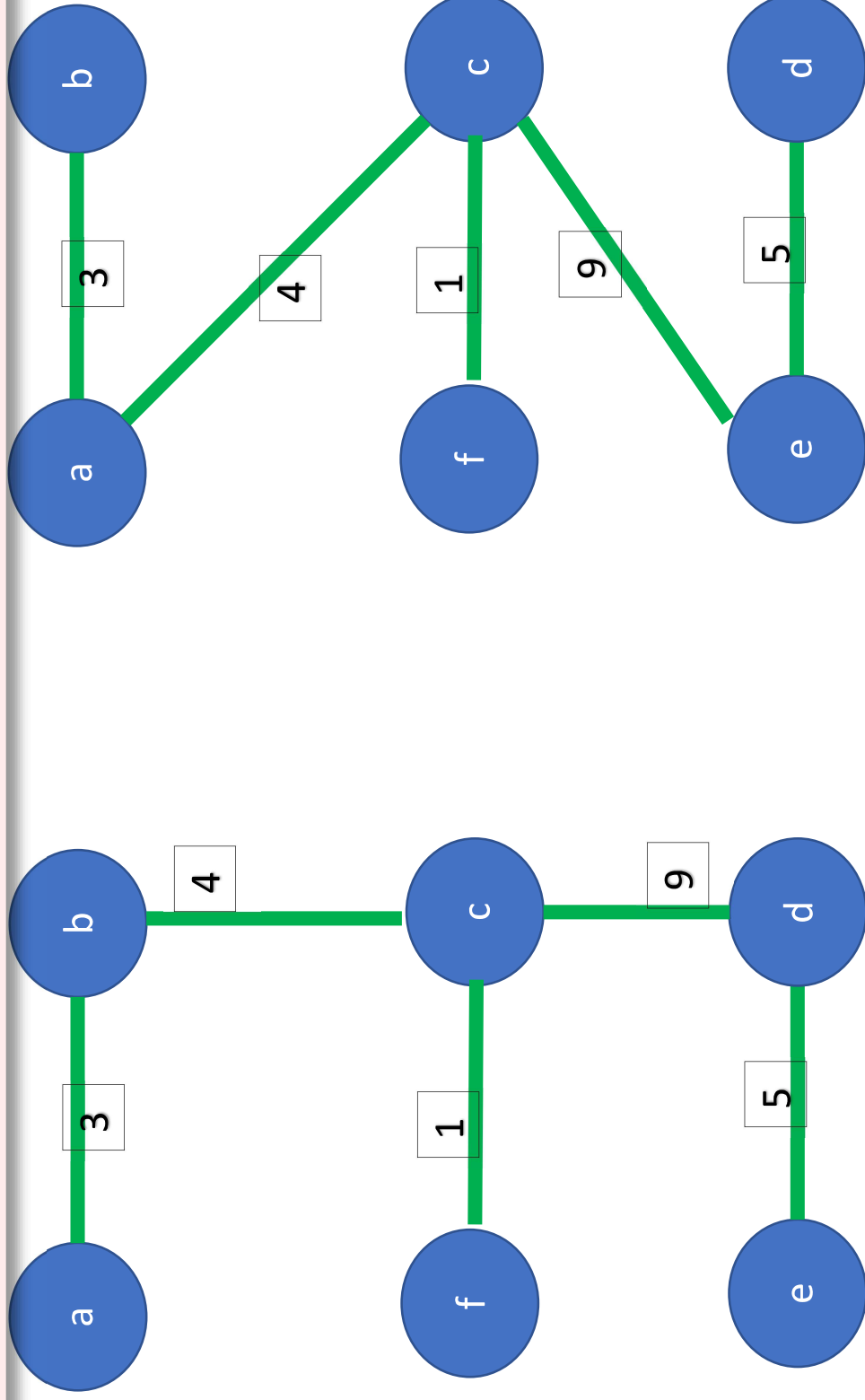


Poids de l'arbre couvrant = $1+3+4+5+9=22$

Algorithme de Kruskal (comparer les deux exécutions)

Conclusion

Le poids de l'arbre couvrant est toujours le même mais les arbres couvrants sont différents.

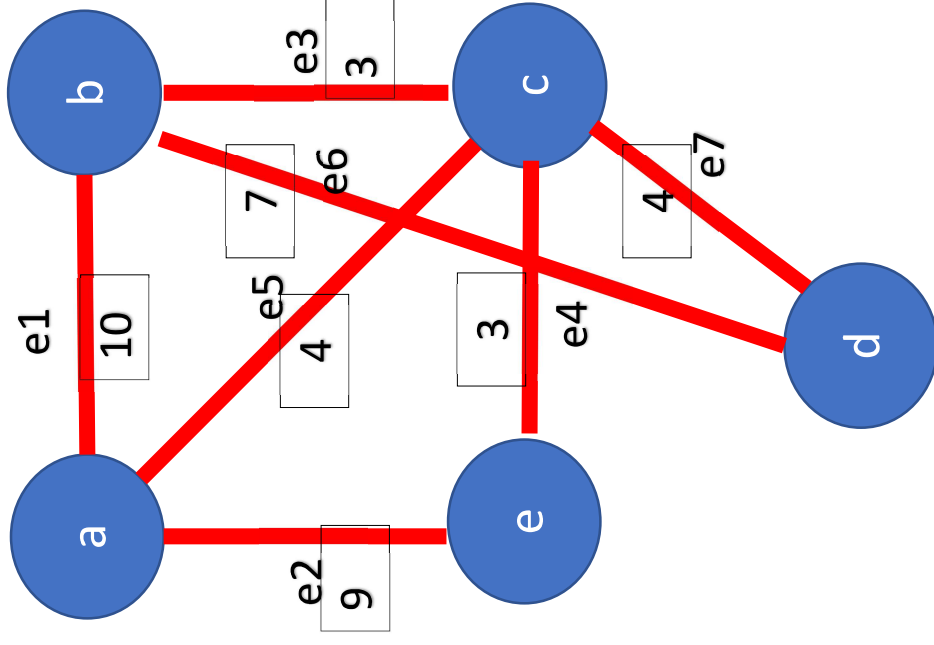


Algorithme de Kruskal (exercice)

- Construire un arbre couvrant de poids minimal

Exemple:

Un graphe connexe et pondéré $G = (V, E)$ défini par l'ensemble fini de sommets $V = \{a, b, c, d, e\}$ et par l'ensemble fini des arêtes valuées $E = \{e1, e2, e3, e4, e5, e6, e7\}$ avec les poids respectives $\{10, 9, 3, 3, 4, 7, 4\}$



Principe de l'algorithme de Prim

- Choisir un sommet quelconque.
- Choisir l'arête de poids minimale parmi les arêtes qui sortent du sommet choisi.
- Ajouter les sommets et leurs arêtes incidentes à l'arbre couvrant si pas de création de cycle.
- S'arrêter si le nombre de sommets de l'arbre couvrant atteint le nombre de sommets de l'arbre d'origine.

Prim maintient un arbre acyclique à chaque rajout de sommet

...

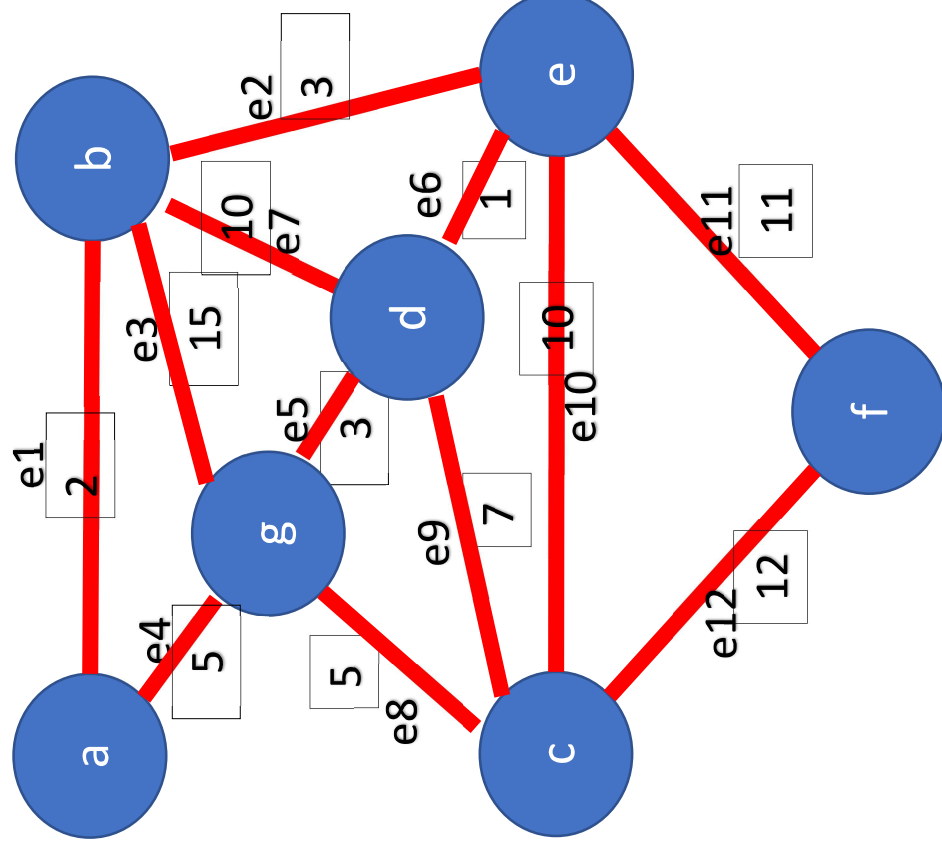
Algorithme de Prim

- Construire un arbre couvrant de poids minimal

donnée:

Un graphe connexe et pondéré $G = (V, E)$ défini par l'ensemble fini de sommets $V = \{a, b, c, d, e, f, g\}$ et par l'ensemble fini des arêtes valuées

$E = \{e_1, e_2, e_3, e_4, e_5, e_6, e_7, e_8, e_9, e_{10}, e_{11}, e_{12}\}$ avec les poids respectives $\{2, 3, 15, 5, 3, 1, 10, 5, 7, 10, 11, 12\}$

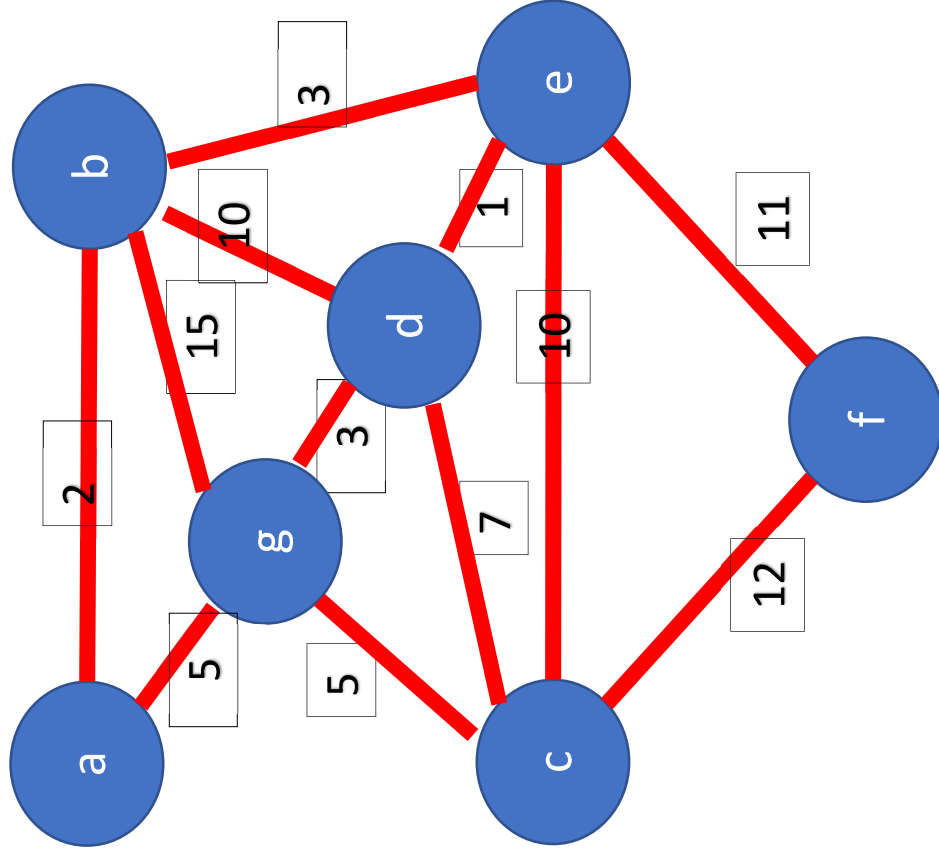


Algorithme de Prim

- Construire un arbre couvrant de poids minimal

donnée:

Un graphe connexe et pondéré $G = (V, E)$ défini par l'ensemble fini de sommets $V = \{a, b, c, d, e, f, g\}$ et par l'ensemble fini des arêtes valuées $E = \{e_1, e_2, e_3, e_4, e_5, e_6, e_7, e_8, e_9, e_{10}, e_{11}, e_{12}\}$ avec les poids respectives $\{2, 3, 15, 5, 3, 1, 10, 5, 7, 10, 11, 12\}$

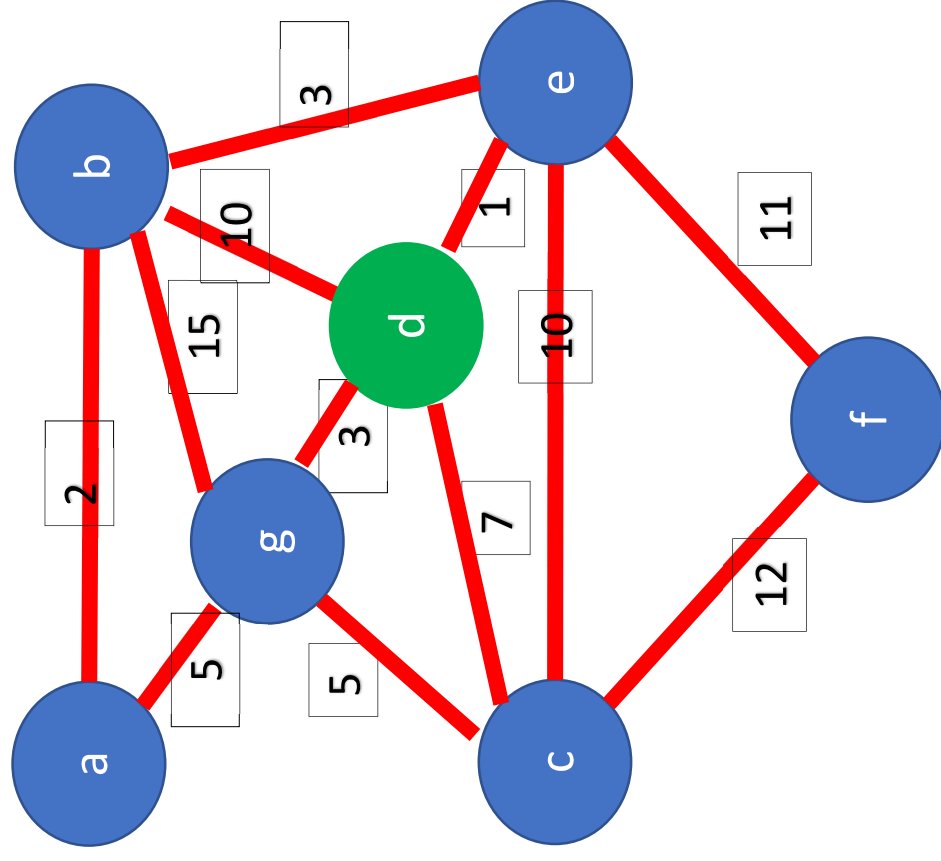


- Construire un arbre couvrant de poids minimal

Étape 1:

Choisir un sommet quelconque

On choisi le sommet d

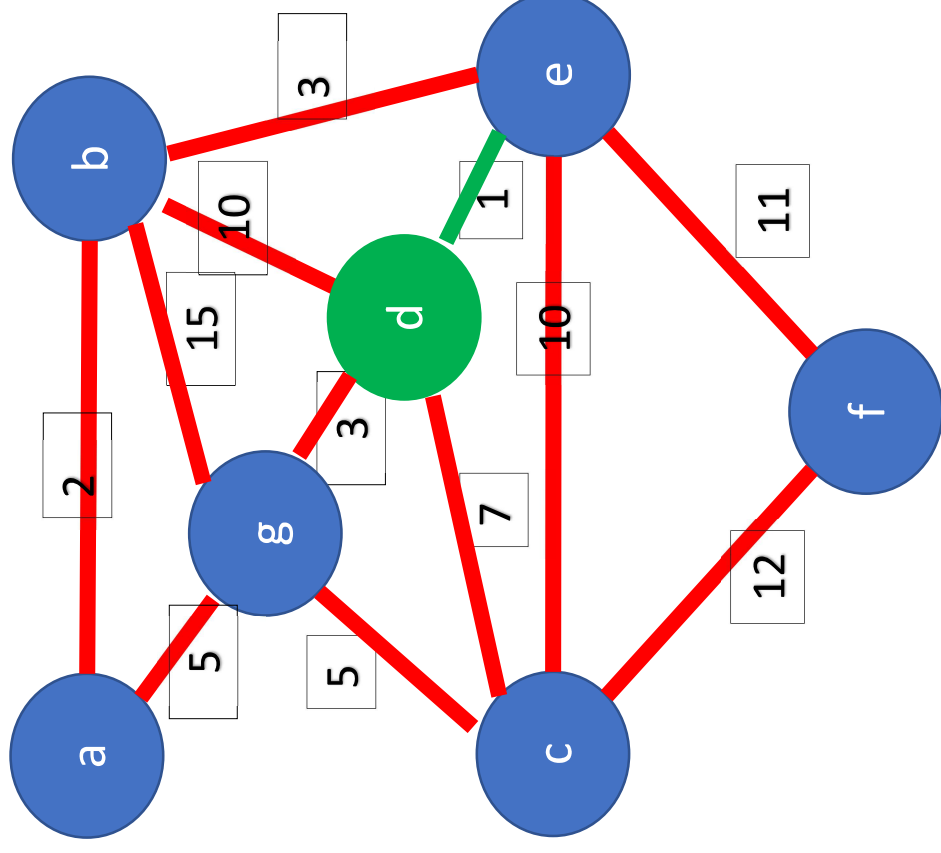


Algorithme de Prim

- Construire un arbre couvrant de poids minimal

Étape 2:
Choisir l'arête de poids minimale
parmi les arêtes qui sortent du
sommet choisi

On choisit l'arête d e de poids = 1

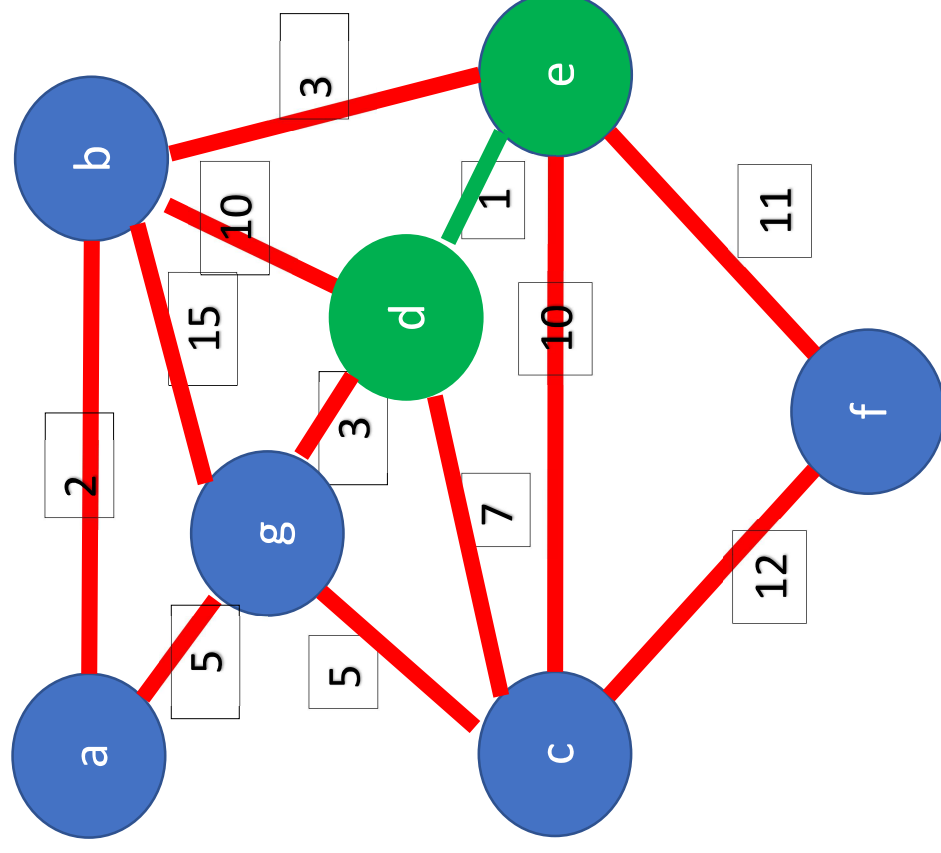


Algorithme de Prim

- Construire un arbre couvrant de poids minimal

Étape 3:
Ajouter le sommet incident à
l'arête choisie

On ajoute le sommet e

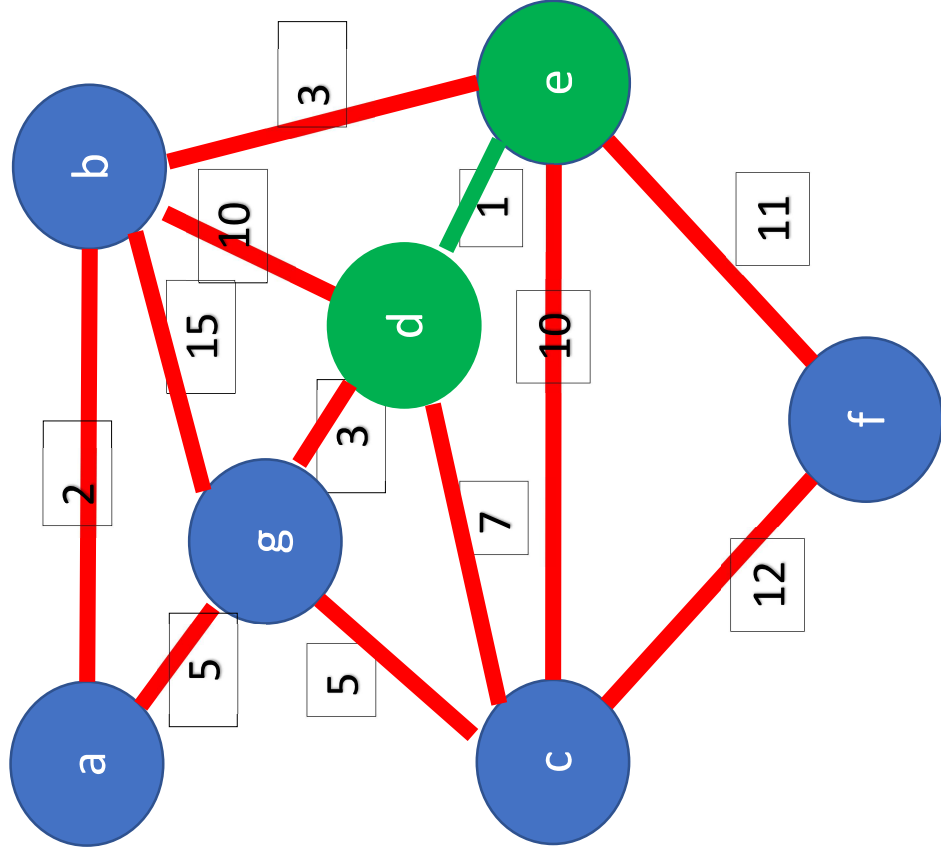


Algorithme de Prim

- Construire un arbre couvrant de poids minimal

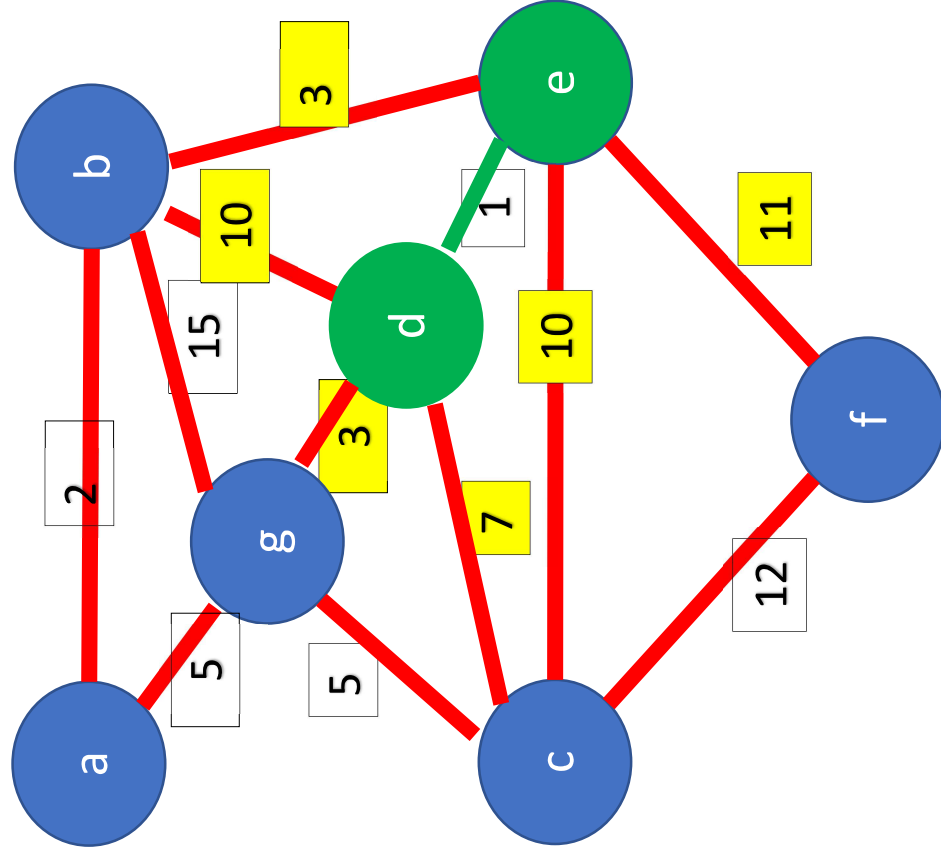
Étape 3:
Ajouter le sommet incident à l'arête choisie

On obtient un graphe partiel contenant deux sommets d et e



- Construire un arbre couvrant de poids minimal

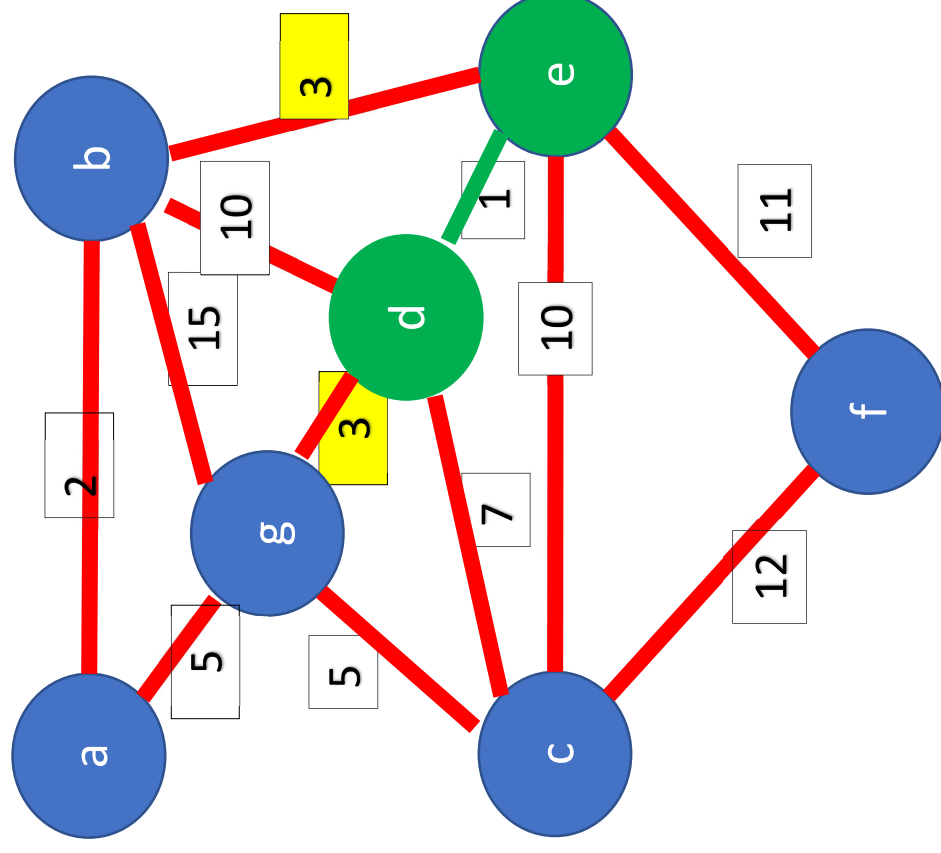
Étape 4:
Considérer toutes les arêtes incidentes à l'arbre obtenu et choisir celle avec le poids minimal



- Construire un arbre couvrant de poids minimal

Étape 4:
Considérer toutes les arêtes incidentes à l'arbre obtenu et choisir celle avec le poids minimal

Il existe deux arêtes de poids minimal b e et g d, choisir celle que vous voulez.

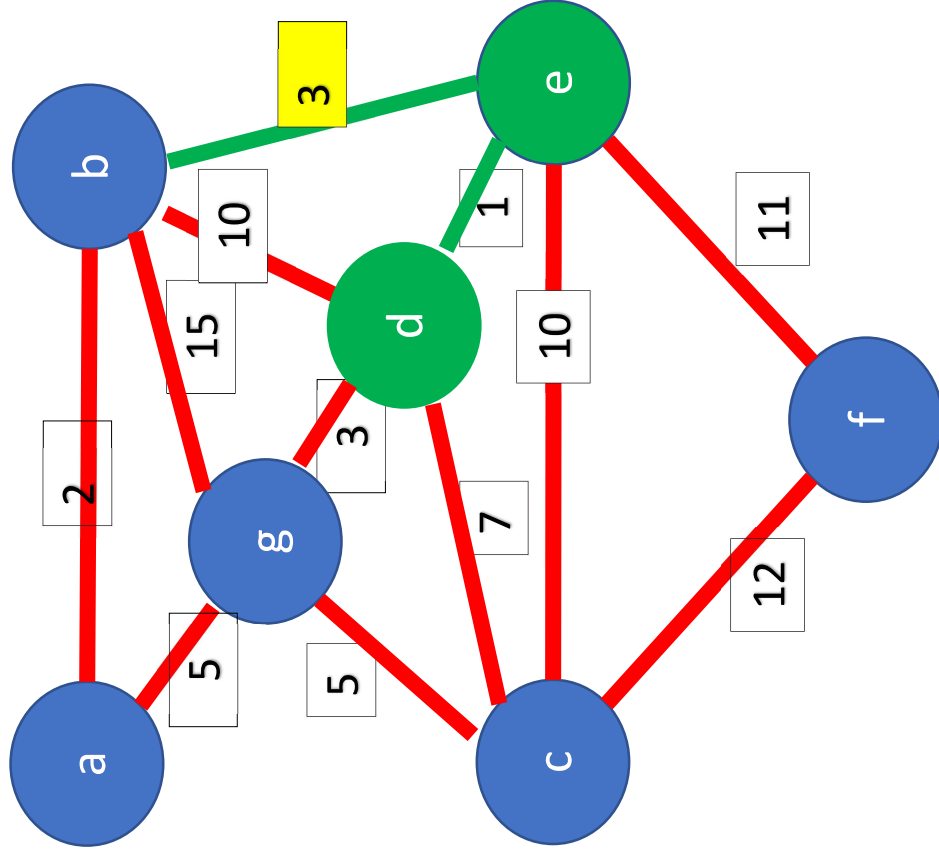


Algorithme de Prim

- Construire un arbre couvrant de poids minimal

Étape 4:
Considérer toutes les arêtes
incidente à l'arbre obtenu et
choisir celle avec le poids minimal

On choisi par exemple l'arête b e.

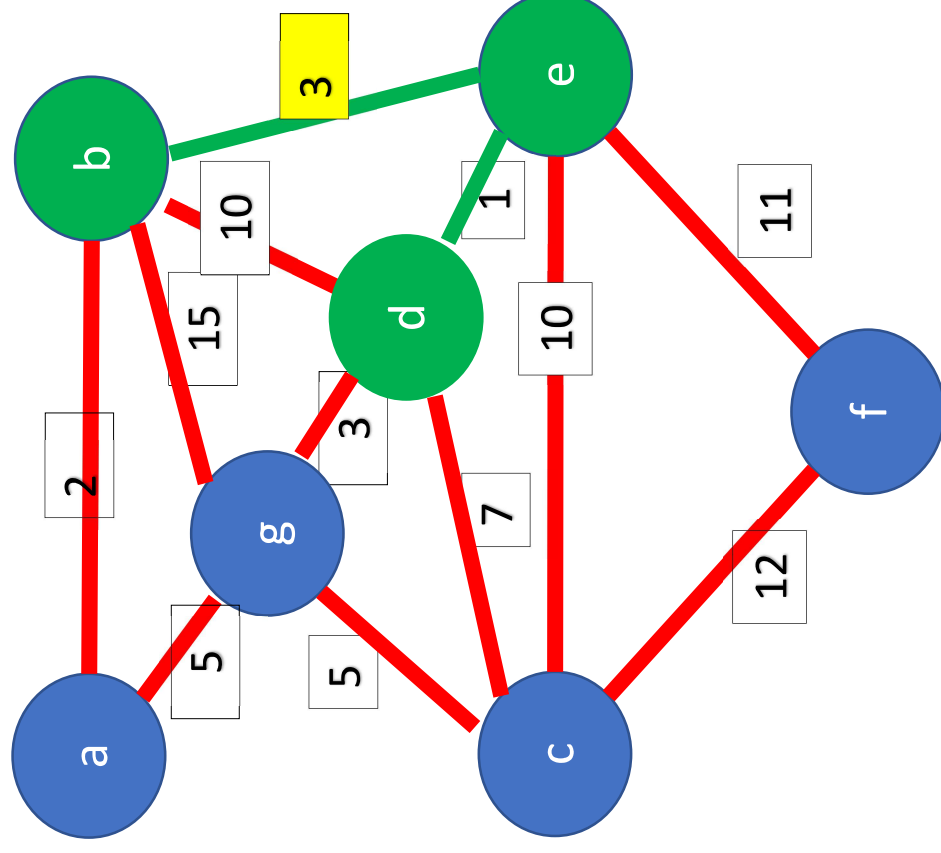


Algorithme de Prim

- Construire un arbre couvrant de poids minimal

Étape 5:
Ajouter le sommet incident à
l'arête choisie

On ajoute le sommet b.



Algorithme de Prim

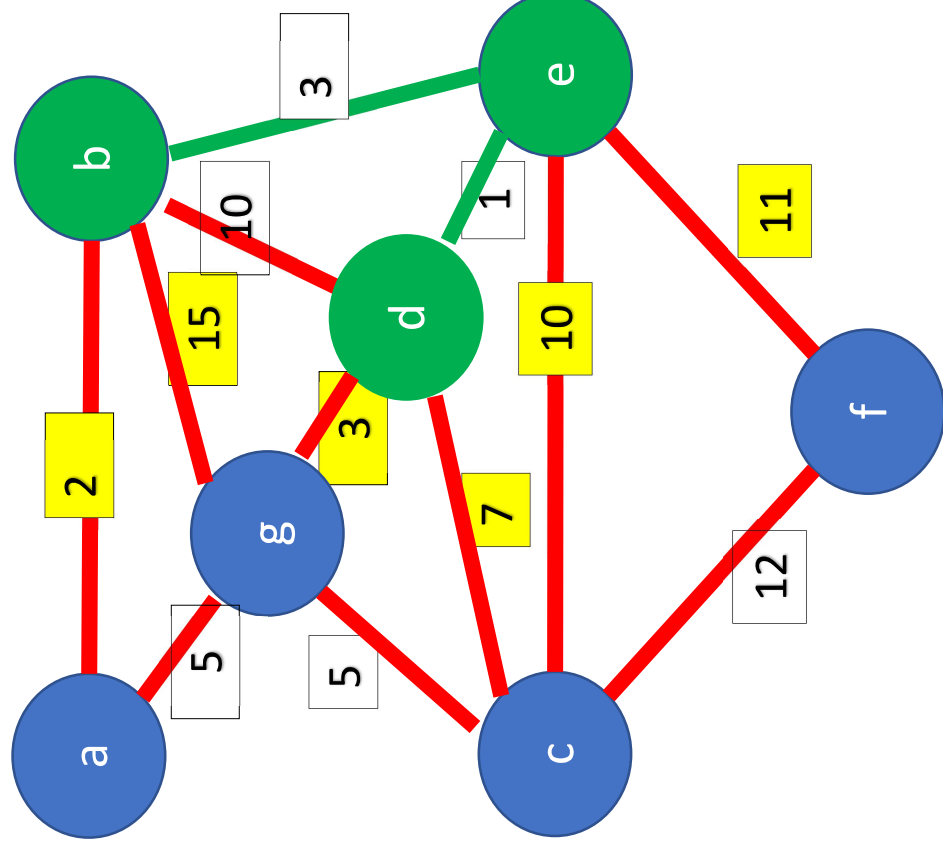
- Construire un arbre couvrant de poids minimal

Étape 6: répéter l'étape 4 et 5

Considérer toutes les arêtes incidentes à l'arbre obtenu qui ont un sommet dans l'arbre et un sommet en dehors de l'arbre, puis choisir celle avec le poids minimal

L'arête bd n'est pas dans la liste des arêtes incidentes à l'arbre obtenu car ses deux sommets sont à l'intérieur de l'arbre.

On choisit l'arête ab de poids 2.

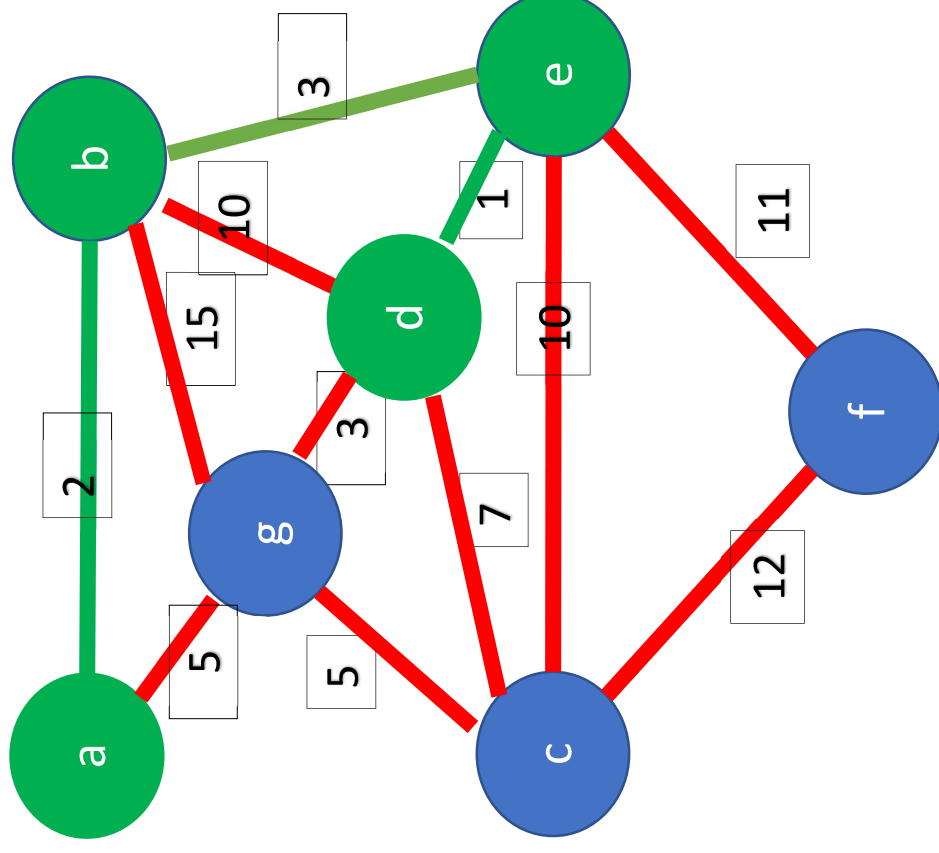


Algorithme de Prim

- Construire un arbre couvrant de poids minimal

Étape 6:
Ajouter le sommet incident à l'arête choisie

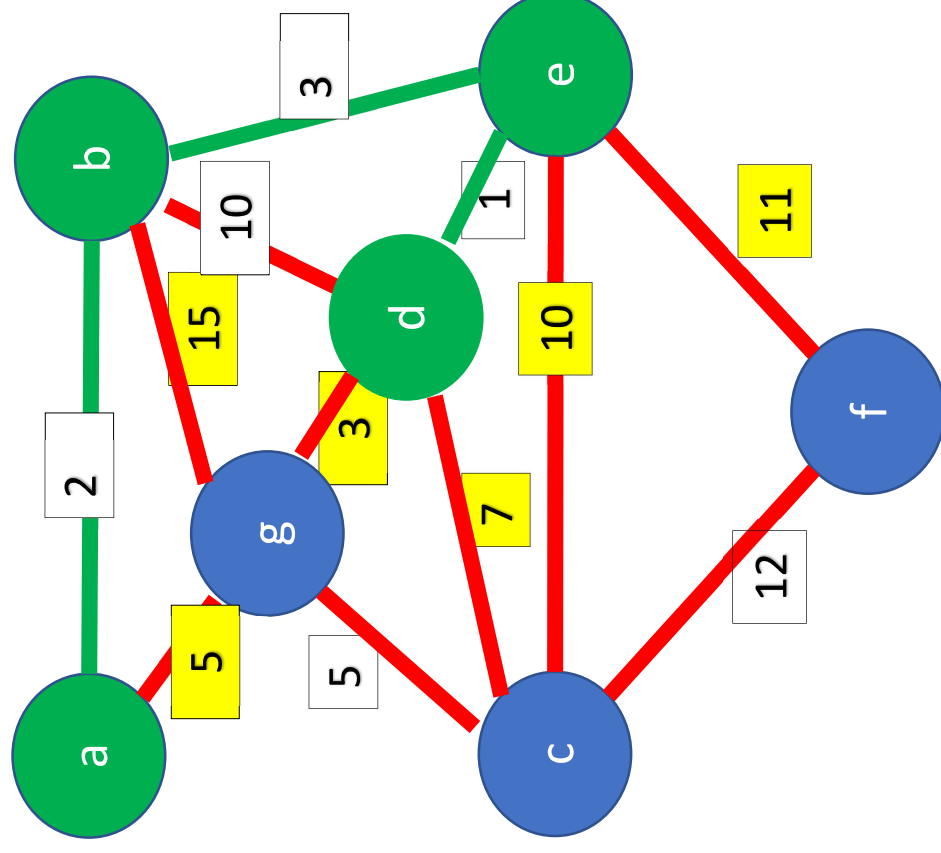
On obtient un graphe partiel contenant quatre sommets d et e et b et a



- Construire un arbre couvrant de poids minimal

Étape 6: répéter l'étape 4 et 5
Considérer toutes les arêtes incidentes à l'arbre obtenu qui ont un sommet dans l'arbre et un sommet en dehors de l'arbre, puis choisir celle avec le poids minimal

On choisit l'arête gd de poids 3 et on ajoute son sommet incident à l'arbre obtenu.

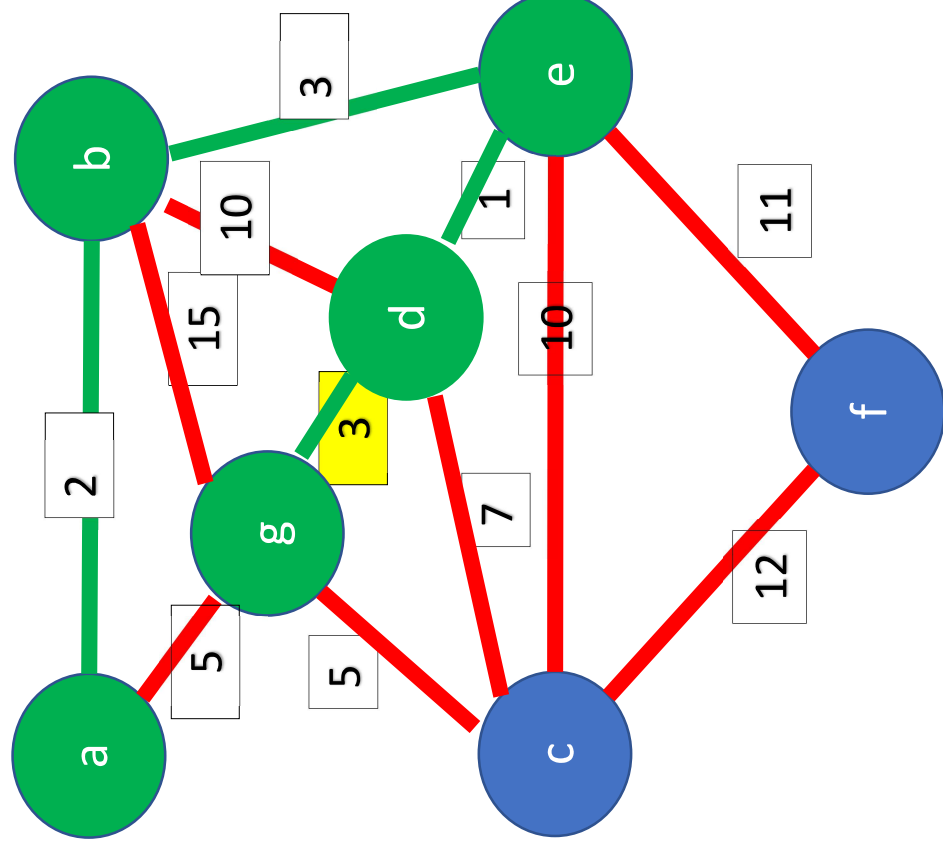


Algorithme de Prim

- Construire un arbre couvrant de poids minimal

Étape 6: répéter l'étape 4 et 5
Considérer toutes les arêtes incidentes à l'arbre obtenu qui ont un sommet dans l'arbre et un sommet en dehors de l'arbre, puis choisir celle avec le poids minimal

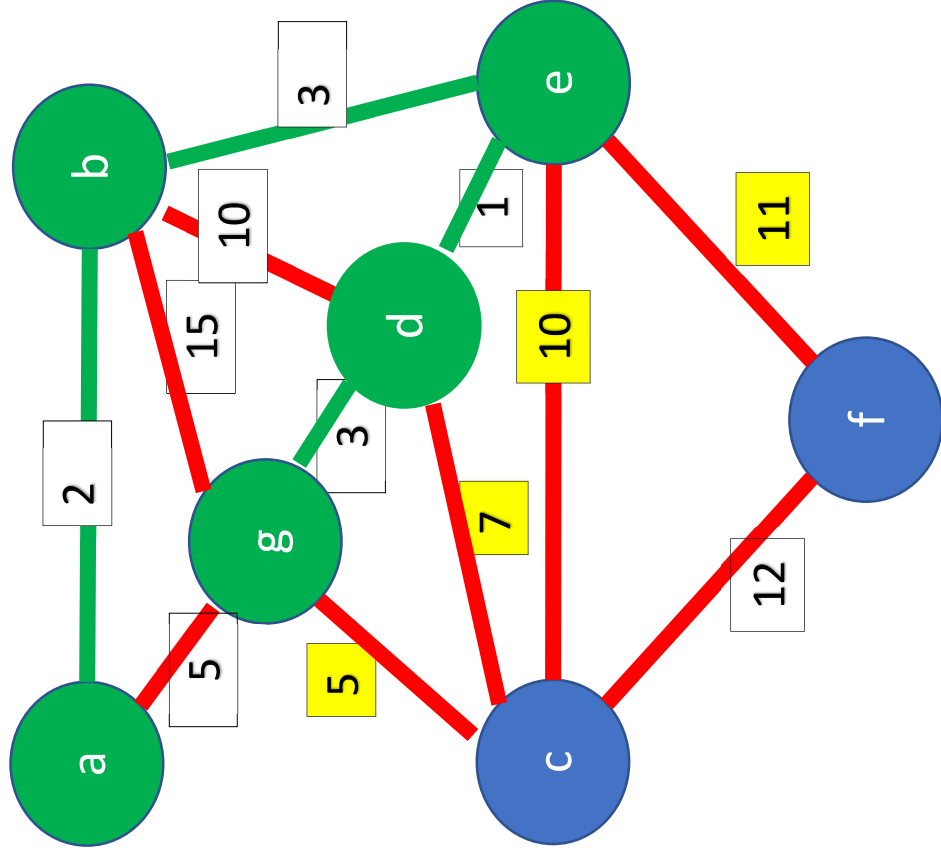
On choisit l'arête gd de poids 3 et on ajoute son sommet incident à l'arbre obtenu.



Algorithme de Prim

- Construire un arbre couvrant de poids minimal

Étape 6: répéter l'étape 4 et 5
Considérer toutes les arêtes
incidentes à l'arbre obtenu qui
ont un sommet dans l'arbre et un
sommet en dehors de l'arbre, puis
choisir celle avec le poids minimal

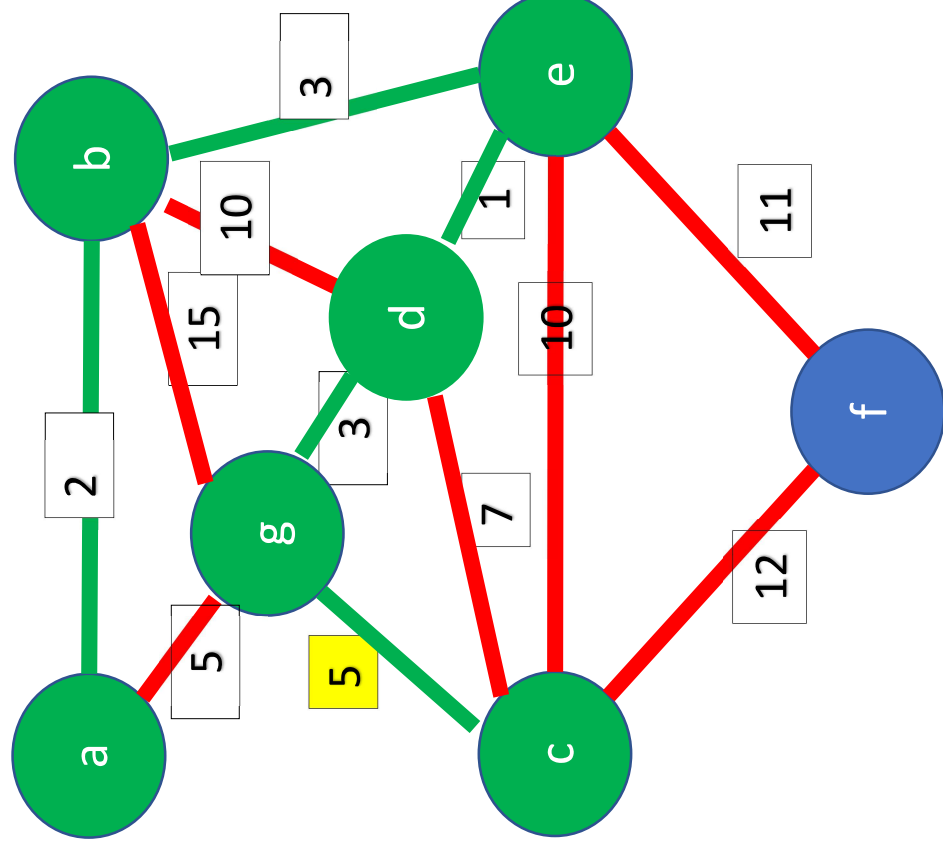


Algorithme de Prim

- Construire un arbre couvrant de poids minimal

Étape 6: répéter l'étape 4 et 5
Considérer toutes les arêtes incidentes à l'arbre obtenu qui ont un sommet dans l'arbre et un sommet en dehors de l'arbre, puis choisir celle avec le poids minimal

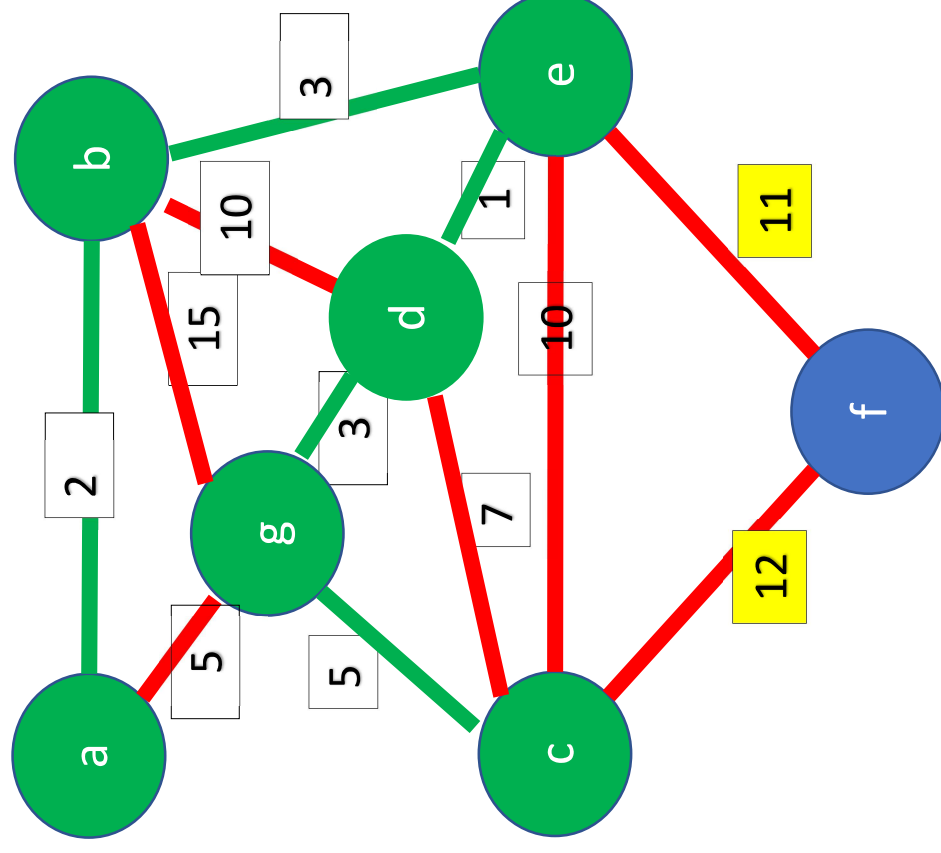
On choisit l'arête gc de poids 5 et on ajoute son sommet incident à l'arbre obtenu.



Algorithme de Prim

- Construire un arbre couvrant de poids minimal

Étape 6: répéter l'étape 4 et 5
Considérer toutes les arêtes incidentes à l'arbre obtenu qui ont un sommet dans l'arbre et un sommet en dehors de l'arbre, puis choisir celle avec le poids minimal

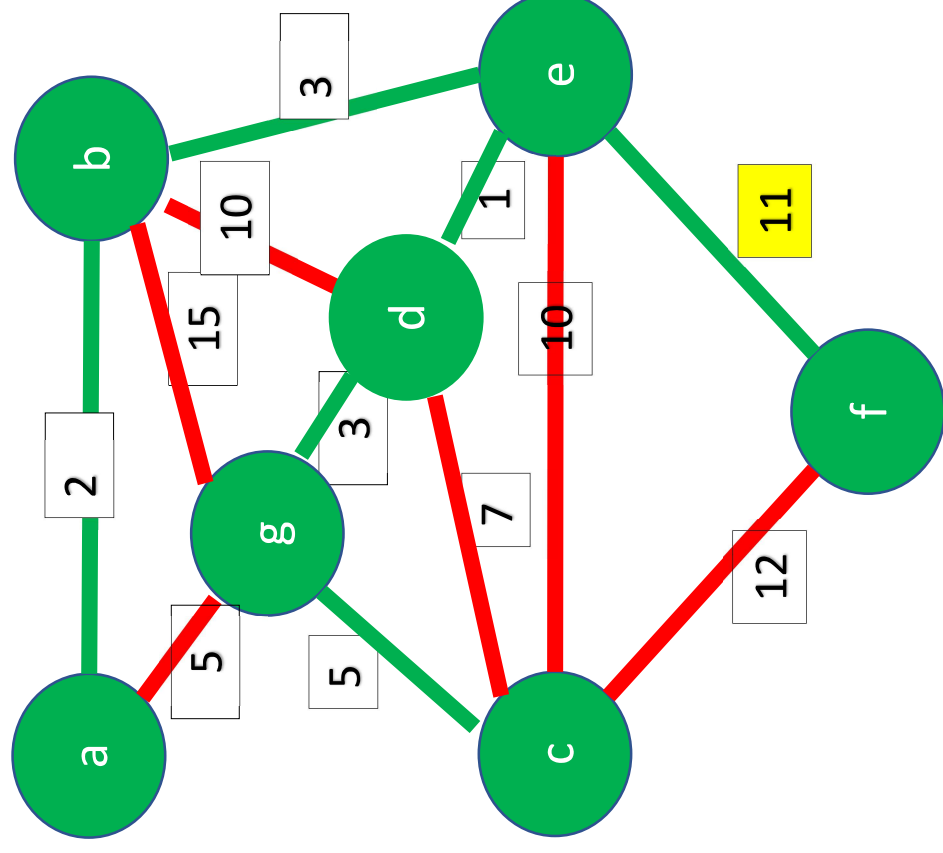


Algorithme de Prim

- Construire un arbre couvrant de poids minimal

Étape 6: répéter l'étape 4 et 5
Considérer toutes les arêtes incidentes à l'arbre obtenu qui ont un sommet dans l'arbre et un sommet en dehors de l'arbre, puis choisir celle avec le poids minimal

On choisit l'arête ef de poids 11 et on ajoute son sommet incident à l'arbre obtenu.

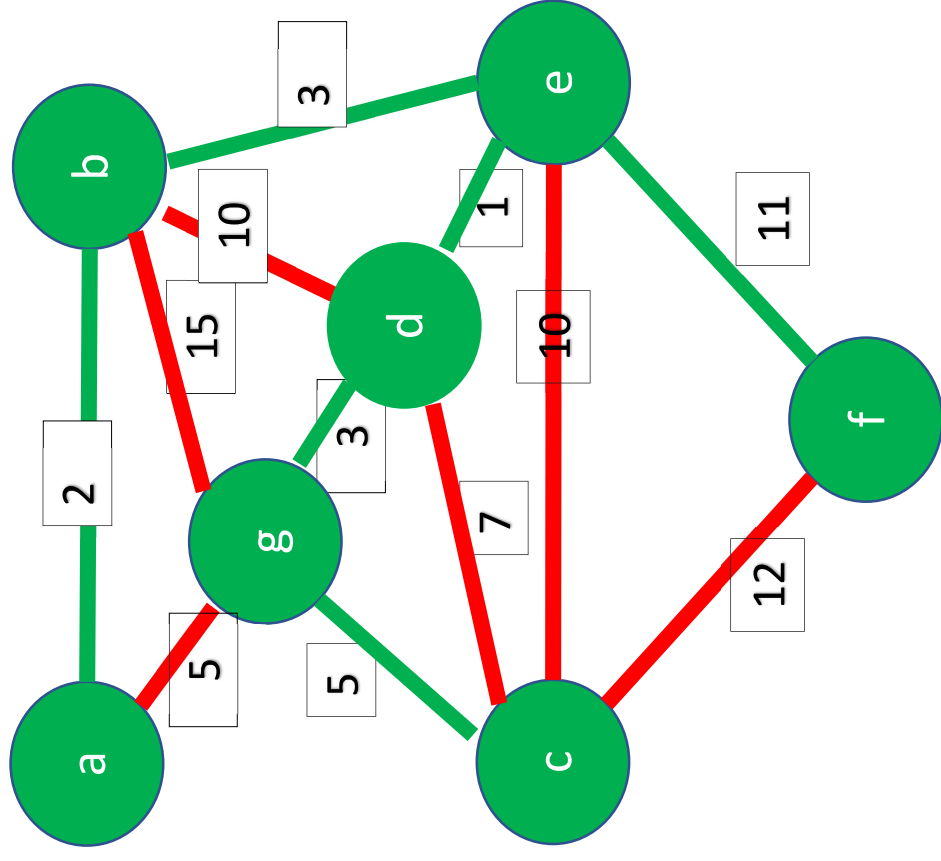


Algorithme de Prim

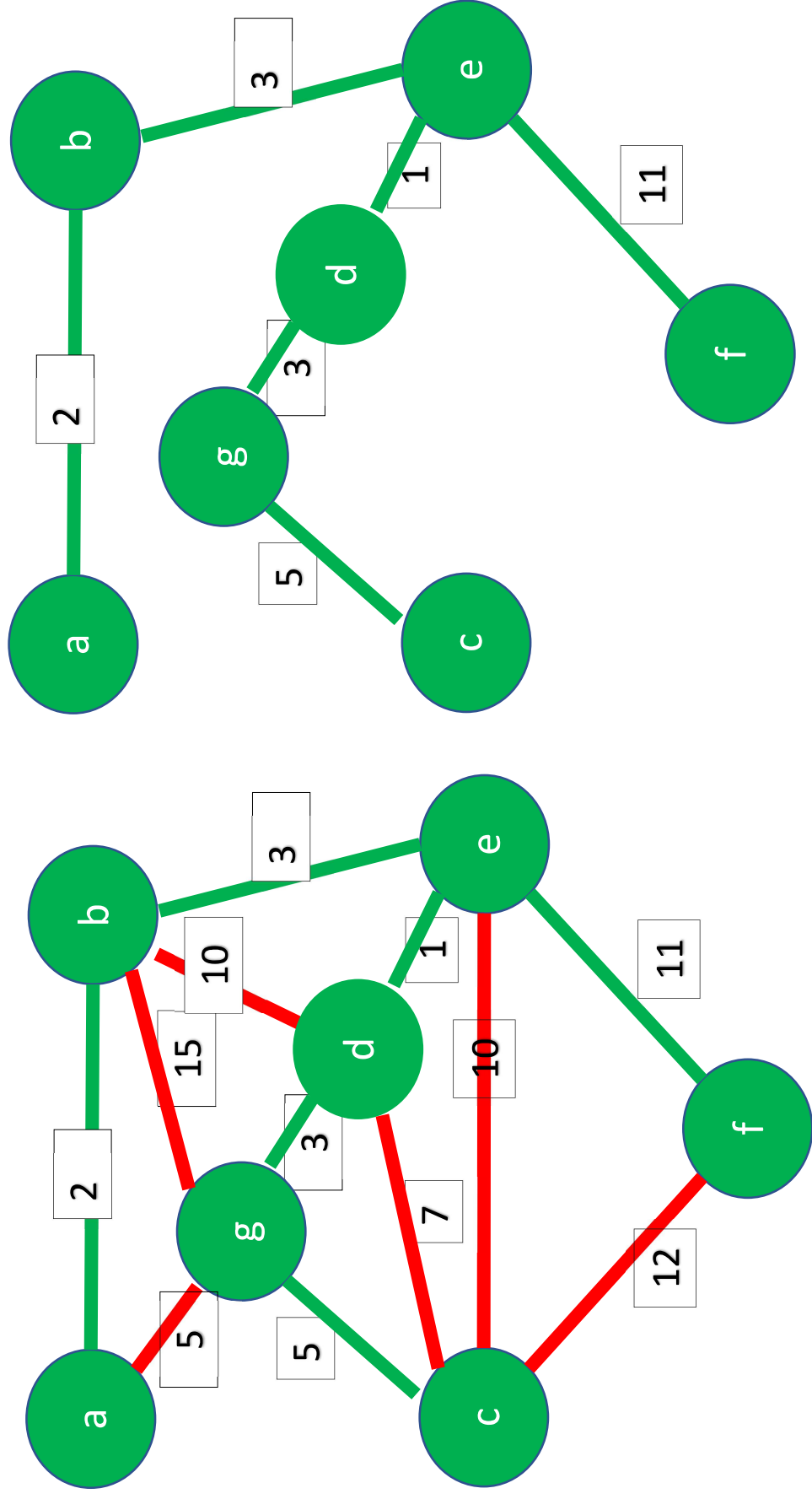
- Construire un arbre couvrant de poids minimal

Étape 7: arrêter si tous les sommets sont ajoutés

Tous les sommets de l'arbre ont été ajoutés donc on s'arrête.



Algorithme de Prim



voici l'arbre couvrant de poids min Poids = $2+3+1+3+5+11=25$.

Quelques liens utiles

Lien 1 :

- <http://www.nymphomath.ch/graphes/graphes.pdf> (Introduction à la théorie des graphes)

Lien 2 :

- <http://www.nymphomath.ch/graphes/corriges.pdf> (Solution des exercices)

Lien 3 :

- https://www.youtube.com/channel/UCHtJVeNLyR1yuJ1_xCK1WRg/playlists (Youtube)

...

- Claude BERGE, Graphes, Edité par Gauthier-Villars (1983), ISBN 10: [2040155554](#) ISBN 13 : [9782040155551](#)
- Bernard ROY, Algèbre moderne et théorie des graphes orientées vers les sciences économiques et sociales, Edité par Dunod Paris, 1969 (volume 1 : Notions et résultats fondamentaux), 1970 (volume 2 : Applications et problèmes spécifiques).
- Gondran et Minoux, Graphes et algorithmes, Edité par Eyrolles Paris, 1978.